



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

A C++ Thread Pool for Multi-threaded Acceleration in fdaPDE

Project Report for the course
Advanced Programming for Scientific Computing

Author: **Guido Maria Poletti, Simone Solidoro**

Student ID: 10718403, 10656115
Advisor: Prof.ssa Laura Maria Sangalli
Co-advisors: Dott. Alessandro Palummo
Academic Year: 2024-25

Contents

Contents	i
Introduction	1
1 Fundamentals of Concurrent Programming in C++	3
1.1 Multi-core architectures	3
1.2 Multithreading in C++	5
1.2.1 <code>std::atomic</code> and <code>std::memory_order</code>	13
1.2.2 Thread-safe data structures	19
2 Thread pool implementation	21
2.1 Introduction	21
2.2 <code>synchro_queue</code>	23
2.2.1 <code>synchro_queue</code> template class	23
2.2.2 Common API	25
2.2.3 <code>access_model::relaxed</code>	26
2.2.4 <code>access_model::deferred</code>	30
2.2.5 <code>access_model::blocking</code>	39
2.3 Thread pool	42
2.3.1 <code>worker</code> class	43
2.3.2 <code>threadpool</code> class	45
2.3.3 Implementation details	49
2.3.4 Scheduling and Stealing strategies	53
2.3.5 Synchronization of <code>count_job_</code>	57
2.3.6 <code>parallel_for</code>	58
3 Performance testing	65
3.1 Ad hoc tests	65
3.1.1 <code>synchro_queue</code> tests	66

3.1.2	<code>threadpool</code> tests	72
3.2	Scalability tests	74
3.2.1	Introduction to scalability testing	74
3.2.2	<code>parallel_for</code> tests	75
3.2.3	Grid Search	79
3.2.4	Matrix assembly	87
3.2.5	Effect of granularity	97
4	Execution instructions	101
4.1	Requirements	101
4.2	Compilation	101
4.3	Execution	102
5	Future developments	103
5.1	Lock-Free <code>synchro_queue</code>	103
5.2	Worker Synchronization: Barriers and Fork-Join	103
5.3	FEM Matrix Assembly	104
	Bibliography	107

Introduction

The fdaPDE library [1] focuses on the statistical analysis of spatial and functional data over complex multidimensional domains enriched with physics-informed information. It integrates statistical models, such as linear regression and functional principal component analysis (ch. 8, [12]), with the physical structure of the underlying phenomenon. This physical information is incorporated through PDE-based penalization, which requires solving the corresponding partial differential equations numerically using the finite element method (FEM) [4, 11]. Given the complexity of the computational domain and the high dimensionality of the problem, the computational cost of the models implemented in fdaPDE can become very high. Moreover, several components of the library naturally lend themselves to parallelization, such as the assembly of FEM discretization matrices or the cross-validation of penalization parameters, like k-fold used in [10] or GCI in [13]. For this reason, we developed from scratch a multithreading framework, leveraging C++20's parallel capabilities, to improve the performance of the most computationally intensive parts of the library.

First, we developed an appropriate structure to store the jobs to be ran in parallel, integrating standard library containers together with mutexes, condition variables and atomic instructions, in order to make the structure thread-safe.

Subsequently, we integrated the queue into a thread pool capable of spawning worker threads to execute the submitted jobs.

Finally, we implemented a parallel for method and used it to parallelize two sections of the fdaPDE library, namely the assembly of matrices for Finite Elements Method, and the search of optimal values on a grid.

Our code was then tested against its sequential counterparts, yielding successful results, proving the scalability and performance of our work.

1 | Fundamentals of Concurrent Programming in C++

1.1. Multi-core architectures

In the last decades computer processors have evolved from single-core designs to multi-core architectures, where multiple processing units, named cores, are integrated on a single chip. Back in the times of single-core architectures, the access to the single CPU was orchestrated, in a time-sharing manner, by the operating system in a way to provide a sense of parallel computation. Nowadays, multi-core architecture enable true parallel processing, significantly broadening the computational performances of applications.

In any computing system, data and program instructions are stored in what is named main memory, commonly known as RAM. However, accessing main memory is relatively slow compared to the processor's operating speed. For instance, if a program, while executing a simple for loop, were to continuously access the main memory, the execution speed of the program would be bounded by the RAM's operating speed, instead of that of the CPU. For this reason, to improve efficiency, processors uses a hierarchy of small but very fast on-chip memories, named caches. These caches form a hierarchical structure, typically organized into multiple levels, referred to as L1, L2, and sometimes L3 cache. The smaller L1 caches are private to each processor, providing fast access to the most frequently used data, typically with an access speed of few clock cycles. Higher-level caches, the L2 and L3 caches, are shared among multiple cores to reduce redundant access to main memory and allow fast inter-core communication in multi-threaded execution. Figure 1.1 gives a schematic example of the memory hierarchy in a multi-core architecture.

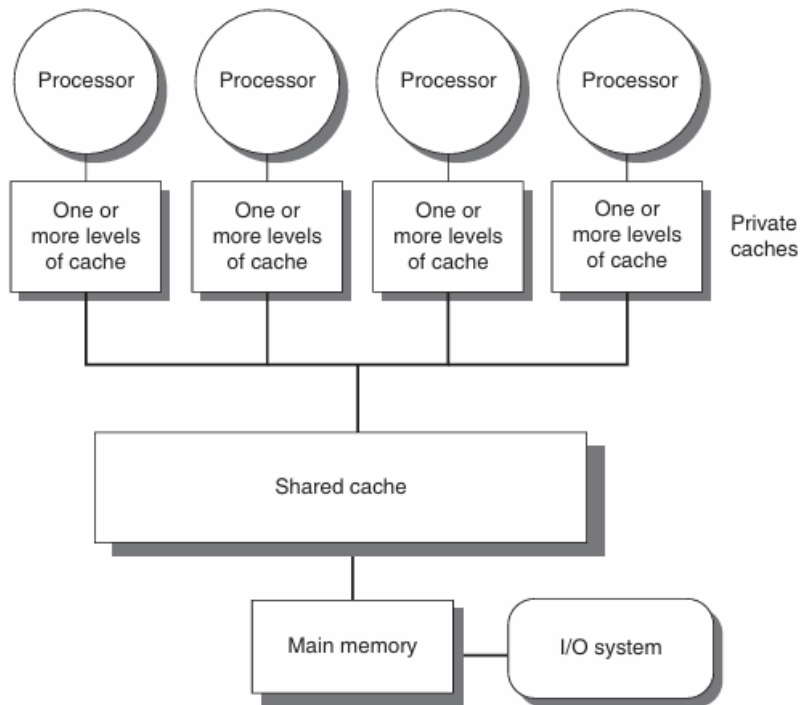


Figure 1.1: Basic structure of a centralized shared-memory multiprocessor based on a multi-core chip. Source: [8].

The presence of a shared global memory and private caches associated with each processor core gives rise to the *cache coherence* problem, namely the possibility that different cores may temporarily observe different values of the same memory location. An illustrative example of this phenomenon is reported in [8]:

Time	Event	Cache contents for processor A	Cache contents for processor B	Memory contents for location X
0				1
1	Processor A reads X	1		1
2	Processor B reads X	1	1	1
3	Processor A stores 0 into X	0	1	0

Figure 1.2: The cache coherence problem for a single memory location (X), read and written by two processors (A and B).

To ensure cache coherence, modern shared-memory systems rely on hardware protocols such as MESI [8]. These protocols associate a coherence state with each cache line, the smallest unit of memory managed by the cache, and guarantee that updates performed by one processor are correctly propagated to all other cached copies of the same memory location.

As a consequence, cache coherence is generally handled transparently by the hardware and does not require explicit intervention from the programmer. However, while coherence protocols preserve correctness, they may introduce non-negligible performance overhead. In particular, unfavorable memory access patterns can lead to phenomena such as *false sharing*, in which independent variables residing on the same cache line are accessed by different threads, causing unnecessary cache invalidations and degraded performance.

While cache coherence mechanisms ensure consistency at the hardware level, parallel execution is ultimately exposed to programmers through software abstractions provided by the operating system. In particular, the operating system defines the execution model through which programs interact with multi-core hardware, introducing abstractions such as processes and threads. A process is an abstraction introduced by the operating system to represent a running program together with its associated address space and system resources. Within a process, one or more threads may exist. Threads are the smallest schedulable units of execution and represent independent sequences of instructions that are executed by the available processor cores. Multiple threads within the same process share code, heap, and global variables, while maintaining private execution contexts such as stacks and registers. Both processes and threads are managed by the operating system and can be scheduled on single-core, multi-core, or multiprocessor systems.

Sharing memory between threads, while avoiding the communication overhead typical of distributed parallelism models, introduces significant risks. In particular, unsynchronised concurrent access by multiple threads to shared memory locations may lead to different execution outcomes depending on the order in which operations are interleaved, a phenomenon known as a *race condition*. More precisely, a *data race* occurs when two or more threads access the same memory location concurrently, and at least one of these accesses is a write, in the absence of proper synchronization. In such situations, the processor is free to reorder or interleave instructions arbitrarily, potentially resulting in *undefined behavior*.

To prevent these issues and ensure the correctness of concurrent execution, the C++ Standard Template Library (STL) provides a set of synchronization primitives that allow safe coordination of access to shared resources. The following sections introduce the main synchronization mechanisms employed in this work.

1.2. Multithreading in C++

The creation of threads in C++ is handled through the `std::thread` class. The code below reports an example of thread creation in C++.

```

#include <thread>
void fun_async(){ /* do something */ };
struct C { void class_method(){ /* do something */ }; };
int main(){
    // create from lambda
    auto fun_lambda = [](){ /* do something */ };
    std::thread t0(fun_async);
    // create from free function
    std::thread t1(fun_lambda);
    // create from class method
    C c;
    std::thread t2(&C::class_method, &c);

    // ... perform other operations while threads execute their work ...

    t0.join(); // wait until t0 terminate execution
    t1.join();
    t2.join();
    return 0;
}

```

The `std::thread` constructor takes a callable object, representing the work to be done asynchronously, and, by interfacing with the operating system, starts a new thread of execution in which the callable is run. At the end of execution, a thread must be explicitly terminated using the `join()` method, which waits for the thread to complete.

From the moment it is created until it is joined, a thread is uniquely identified by its ID (an object of type `std::thread::id`), which can be accessed through the function `std::this_thread::get_id()`.

Other useful functions from the `std::thread` class and the `std::this_thread` namespace, which operate directly on the calling thread, are:

- `std::this_thread::yield()`: temporarily suspends the thread, yielding CPU resources to allow other threads to run
- `std::thread::hardware_concurrency()`: returns the maximum number of hardware threads available on the system.

While the threads are executiong, in order to avoid a race condition, is fundamental that accesses to shared memory are properly synchronized among threads. Such synchronization is achieved through primitives provided by the STL. Here, we introduce the main con-

cepts of `std::mutex`, `std::condition_variable`, `std::future`, `std::packaged_task`, `std::atomic<>` and `memory_order`. Afterwards, we introduce a brief classification of what are commonly referred to as thread-safe structures. The reader already familiar with these concepts can skip to Section 2.1.

`std::mutex`

A `std::mutex` is an object providing exclusive access to a shared memory region, defined in the `<mutex>` header. To enter the critical section protected by the mutex, a thread must acquire the mutex using the `lock()` method. Only one thread at a time can hold the lock on a mutex. Once the critical section is completed, the thread must release the mutex using the `unlock()` method, allowing other threads to acquire it. In this way, the mutex ensures that concurrent accesses are performed safely by serializing operations on shared data.

The following code snippet shows an example of the use of a `std::mutex` to safely operate on shared data:

```
#include <iostream>
#include <thread>
#include <mutex>
struct shared_data { /* ... */ };
std::mutex m;
shared_data s_d;
void fun() {
    m.lock();
    // only one thread can execute this block at a time
    // --- critical section ---
    // modify shared_data here
    // invariants of shared_data may be temporarily broken
    m.unlock(); // release the mutex, allowing other threads to acquire it
}
int main() {
    std::thread t1(fun);
    std::thread t2(fun);
    t1.join();
    t2.join();
}
```

The use of this primitive also introduces certain risks: a thread attempting to acquire a mutex that is already locked will remain blocked until the lock becomes available. If, due to a programming error, waiting threads remain blocked indefinitely, the program may be prevented from terminating. This situation is known as a *deadlock* (see [14], Ch. 3.2.4). An example of a deadlock is shown below:

```
#include <iostream>
#include <thread>
#include <mutex>

std::mutex m1;
std::mutex m2;
void m1_m2() {
    m1.lock();
    m2.lock();
    // --- critical section ---
    m2.unlock();
    m1.unlock();
}
void m2_m1() {
    m2.lock();
    m1.lock();
    // --- critical section ---
    m1.unlock();
    m2.unlock();
}
int main() {
    std::thread t1(m1_m2);
    std::thread t2(m2_m1);
    // Potential deadlock scenario:
    // t1 acquires m1, t2 acquires m2
    // t1 tries to acquire m2 but it's already locked by t2 -> waits
    // t2 tries to acquire m1 but it's already locked by t1 -> waits
    // both threads are blocked forever -> deadlock
    t1.join();
    t2.join();
}
```

As one might expect, the risk of deadlock increases with the number of mutexes ac-

quired simultaneously. To mitigate this problem, in addition to following programming best practices, such as acquiring multiple mutexes in the same order across threads, the C++ standard library provides classes that, through the RAII (Resource Acquisition Is Initialization) pattern, ensure proper mutex release. Two examples of such classes are:

- `std::lock_guard`, which acquires the mutex upon construction and automatically releases it upon destruction
- `std::unique_lock`, which allows more flexible management by enabling the mutex to be unlocked and relocked multiple times during the lifetime of the object.

```
#include <mutex>
#include <thread>

struct shared_data{/*...*/};
std::mutex m;

void atomic_modification(shared_data & sd){
    std::lock_guard<std::mutex> l(m);
    /*critical section*/
    // RAII pattern,
    // lock realise by the destructor when l goes out-of-scope
}

void modification(shared_data & sd){
    std::unique_lock<std::mutex> l(m);
    /*critical section*/
    l.unlock();
    /*...*/
    l.lock();
    /*another critical section*/
    l.unlock();//optional
}

int main(){
    shared_data SD;
    std::thread t1(atomic_modification,std::ref(SD));
    std::thread t2(modification,std::ref(SD));
    t1.join();
    t2.join();
    return 0;
}
```

```
}
```

There are different types of mutexes. In this work, we also use `std::shared_mutex`, which allows acquiring the lock in two different modes:

- Write mode: only one thread can acquire the lock at a time, and while it holds the lock, no other acquisitions are allowed (like a classic lock).
- Read mode: multiple threads can acquire the lock simultaneously; this is intended for threads that only need to read from the critical section and therefore do not cause *race conditions*.

`std::condition_variable`

To synchronize operations occurring within critical sections, `std::condition_variable` objects are used, defined in the `<condition_variable>` header. After acquiring a mutex lock, they allow checking whether a certain condition holds; if it does not, they release the mutex and suspend the execution of the thread until another thread, presumably one that has made the condition true, notifies the suspended thread of the possible change. Once notified, the suspended thread, reacquire the mutex, and checks the condition again. If the condition is satisfied it proceeds, otherwise it is suspended again. This mechanism is implemented through the methods of the `std::condition_variable` class (See [14], Ch 4.1.1):

- `template<class Predicate>`
`void wait(std::unique_lock<std::mutex>& lock, Predicate pred):` first evaluates the predicate while holding the mutex; if the condition is `false`, the calling thread releases the mutex and suspends execution until it is notified. Upon notification, the mutex is reacquired and the predicate is re-evaluated. The method returns only when the predicate evaluates to `true`.
- `void notify_one():` wakes up a single thread waiting on the condition variable, provided that the thread was already suspended at the time of the notification.
- `void notify_all():` wakes up all threads waiting on the condition variable. Each awakened thread will compete to reacquire the mutex and recheck the condition.

The notification should not be triggered while holding the mutex lock; in fact, it is considered good practice, whenever possible, to send it after releasing the lock. This helps avoid a situation where the notified thread finds the mutex still locked upon waking.

The following code shows an example of the use of a condition variable:

```

#include <thread>
#include <mutex>
#include <condition_variable>
#include <iostream>
// shared structure between threads, modified concurrently
struct shared_data{
    bool ready = false;
    int value = 0;
    std::mutex m;
    std::condition_variable cv;
};

void produce(shared_data& SD){
    // critical section
    std::lock_guard<std::mutex> l(SD.m);
    SD.ready = true;
    SD.value = 10;
    std::cout<<"producer. value set to"<<SD.value<<std::endl;
    // mutex released by the RAII pattern
    SD.cv.notify_one(); // notify a suspended thread, if any
}

void consume(shared_data& SD){
    std::unique_lock<std::mutex> l(SD.m);
    // wait until notified and ready==true
    SD.cv.wait(l, [&]() {return SD.ready;});
    /* read and use the value produced by produce */
    std::cout<<"consumer. value = "<<SD.value<<std::endl;
}

int main(){
    shared_data sd;
    std::thread t_cons (consume, std::ref(sd));
    // ensures that consume reaches the condition variable before produce
    std::this_thread::sleep_for(std::chrono::milliseconds(1));
    std::thread t_prod (produce, std::ref(sd));
    t_prod.join();
    t_cons.join();
    return 0;
}

```

Only possible output:

```
producer. value set to 10
consumer. value = 10
```

`std::future`

To retrieve the result of an asynchronous operation, the `std::future<>` template class is used, defined in the `<future>` header. A callable object that returns a value of type `T` can store its result in a `std::future<T>`. When the asynchronous operation completes, the future is set to the ready state, and the produced value can be waited for and obtained using the `get()` method, which blocks the calling thread until the result becomes available. We show an example using `std::async`, which, similarly to `std::thread`, takes a callable object together with any required arguments. Unlike `std::thread`, however, `std::async` returns a `std::future` that is associated with the result of the asynchronous operation and allows the caller to retrieve it once the computation has completed. An example of its usage is now shown:

```
#include <future>

int async_reduce(std::vector<int>& v){
    int sum = 0;
    for(size_t i = 0; i<v.size(); i++){
        sum += v[i];
    }
    return sum;
}

int main(){
    std::vector<int> v;
    /*populate v*/
    std::future<int> result = std::async(async_reduce,std::ref(v));
    /*do other calculations*/
    int reduce = result.get(); //wait untile async_reduce completes
    /*use reduce value*/
    return 0;
}
```


`std::packaged_task`

The template class `std::packaged_task<>`, defined in the `<future>` header, associates a callable object with a `std::future`, which will hold the return value of the callable. When the `packaged_task` is invoked, the associated callable object is executed and, once it finishes, its result is stored in the associated future. The `std::future` associated with the `std::packaged_task` can be obtained using the `get_future()` method. The following example demonstrates its usage:

```
#include <thread>
#include <future>
double task_fun(int a, char b){/*async task*/}
int main(){
    std::packaged_task<double(int,char)> task(task_fun);
    std::future<double> fut = task.get_future();
    std::thread t (std::move(task),10,'a');
    /*do other calculations*/
    double result = fut.get();
    return 0;
}
```

1.2.1. `std::atomic` and `std::memory_order`

In a multithreaded context, all threads must agree on a unique order of modifications performed on the same object from its creation until the end of the program. This total order of write operations is referred to as the *modification order* of the object. By contrast, operations involving different objects are not required to follow a single global ordering shared by all threads. As a result, in the absence of explicit synchronization, there is no guarantee that different threads will observe memory operations in the same order. This lack of a global ordering is a consequence of the execution model adopted by modern processors. CPUs are not required to execute instructions in the order specified by the program, but only to preserve the correctness of the final result according to the language memory model. In order to maximize throughput and efficiently utilize hardware resources, processors are free to reorder instructions whenever such reordering does not affect the observable behavior of a single-threaded program. As discussed in Section 1.1, accesses to main memory are significantly slower than register or cache operations. When an instruction depends on data that must be fetched from main memory, the processor may avoid stalling the entire execution pipeline by issuing and executing subsequent in-

dependent instructions. If no unresolved data dependencies exist, these later instructions may be completed and committed before earlier ones. This execution strategy is known as *out-of-order execution* and is employed by all modern processors due to the substantial performance benefits it provides. While out-of-order execution is transparent and safe in sequential programs, it introduces subtle issues in concurrent applications. The main source of these issues lies in the fact that high-level programming statements are generally not atomic. As an illustrative example, consider the following simple C++ program:

```
void increment(int* p) {
    for (int i = 0; i < 1000000; ++i) {(*p)++;}
}
int main() {
    int value = 0; // shared state
    increment(&value);
    return 0;
}
```

The assembly code generated by the compiler is shown below:

```
increment(int*):
    push    rbp
    mov     rbp, rsp
    mov     QWORD PTR [rbp-24], rdi
    mov     DWORD PTR [rbp-4], 0
    jmp     .L2
.L3:
    mov     rax, QWORD PTR [rbp-24]
    mov     eax, DWORD PTR [rax]    !!!
    lea     edx, [rax+1]            !!!
    mov     rax, QWORD PTR [rbp-24] !!!
    mov     DWORD PTR [rax], edx
    add     DWORD PTR [rbp-4], 1
.L2:
    cmp     DWORD PTR [rbp-4], 999999
    jle     .L3
    nop
    nop
    pop     rbp
    ret
main:
    push    rbp
    mov     rbp, rsp
    sub     rsp, 16
    mov     DWORD PTR [rbp-4], 0
    lea     rax, [rbp-4]
    mov     rdi, rax
    call    increment(int*)
    mov     eax, 0
    leave
    ret
```



a single C++ increment instruction, `(*p)++`, corresponds to three assembly operations

Although the statement `(*p)++` appears as a single operation at the source-code level, it is translated by the compiler into multiple machine-level instructions. In particular, the increment operation consists of a sequence of load, modify, and store instructions, each of which may be independently reordered or interleaved with operations from other threads. As a consequence, when such non-atomic operations are executed concurrently by multiple threads without proper synchronization, the observed behavior may differ from the programmers intent. To address these issues, programming languages introduce *memory orderings*, which define the visibility and ordering constraints of memory operations across threads. Memory orders provide a formal mechanism to reason about concurrency and to control the effects of instruction reordering in multithreaded programs, as discussed in Chapter 5.3.3 of [14].

Let us consider a simple example to illustrate the issues related to this situation and to introduce the tools needed to handle them:

Suppose we have a producer thread A that sets an integer variable `i`, initially equal to 0, and a consumer thread B that wants to read the value set by A and, if it is greater than 0, update a boolean variable `result` from `false` to `true`. To coordinate the threads and ensure that thread B reads `i` only after A has modified it, a boolean variable `ready` (initialized to `false`) is used, thread A sets `ready` to `true` once it has updated `i`.

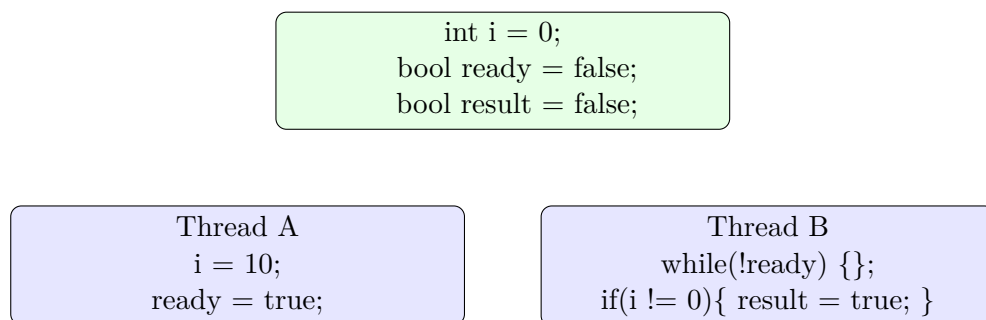


Figure 1.3: Scheme of the example.

Figure 1.3 shows a simple scheme representing this example, highlighting two main problems:

1. The variable `ready` is modified and read by different threads, which can lead to a *data race*.
2. There is no guarantee that thread B will correctly set `result` to `true`, because even after waiting for `ready` to become `true`, there is no assurance that the order of operations observed by thread B is consistent with that of thread A, as it is possible for thread B to see `ready` as `true` while `i` is still 0.

To address these issues, we introduce `std::atomic` variables, which allow us to eliminate data races and ensure proper synchronization between threads.

`std::atomic<>`

As explained in Chapter 5 of [14], atomic variables are variables whose operations are performed atomically, that is, each thread can observe the variable either before or after an operation, but never in an intermediate state. This is guaranteed by the fact that atomic variables are indeed atomic at machine-level, thanks to CPU instructions set capable of performing certain operations with a single machine instruction, therefore avoiding the incurrence of a data race due to instruction reordering. We now show this mechanism by recalling the previous example, where `i` is now made atomic:

```
#include <atomic>

void atomic_increment(std::atomic<int>* p) {
    for (int i = 0; i < 1000000; ++i) {
        p->fetch_add(1, std::memory_order_seq_cst);
    }
}

int main() {
    std::atomic<int> value = 0;
    atomic_increment(&value);
    return 0;
}
```

The assembly code generated by the compiler is shown below:

```
atomic_increment(std::atomic<int>*):
    push    rbp
    mov     rbp, rsp
    mov     QWORD PTR [rbp-40], rdi
    mov     DWORD PTR [rbp-4], 0
    jmp     .L2
.L4:
    mov     rax, QWORD PTR [rbp-40]
    mov     QWORD PTR [rbp-16], rax
    mov     DWORD PTR [rbp-20], 1
    mov     DWORD PTR [rbp-24], 5
    mov     edx, DWORD PTR [rbp-20]
    mov     rax, QWORD PTR [rbp-16]
    lock xadd     DWORD PTR [rax], edx !!!
    nop
    add     DWORD PTR [rbp-4], 1
```

a single C++ increment instruction,
`p->fetch_add(1, std::memory_order_seq_cst),`
corresponds to a single assembly operations

```

.L2:
    cmp     DWORD PTR [rbp-4], 999999
    jle     .L4
    nop
    nop
    pop     rbp
    ret
main:
    push    rbp
    mov     rbp, rsp
    sub     rsp, 16
    mov     DWORD PTR [rbp-4], 0
    lea     rax, [rbp-4]
    mov     rdi, rax
    call    atomic_increment@plt
    mov     eax, 0
    leave
    ret

```

This guarantees the absence of data races.

The main operations that can be performed on atomic variables are:

- `store()`: atomically modifies the value;
- `load()`: atomically reads the value;
- `compare_exchange()`: performs an atomic read–modify–write operation.

This operation is fundamental when working with atomic variables, since a simple sequence of `load()` followed by `store()` would not guarantee correctness: by the time the `store()` is executed, the value read by the previous `load()` might no longer be valid due to concurrent modifications by other threads.

However, the use of atomic variables alone does not generally prevent race conditions, because the order in which multiple threads operate on them is not guaranteed to be unique. Nevertheless, it is possible to introduce an ordering relationship by imposing constraints on instruction reordering through the `memory_order` tags available for atomic operations. There are several tags, but we will only mention those used in this context: `relaxed`, `sequential_consistent`, `acquire`, and `release`.

Next, we introduce the main ordering relations that help us understand thread coordination and form the foundation of the semantics of the different memory orders.

happen-before* , *synchronize-with The *happen-before* relation is defined as follows: if all the effects of an operation A are guaranteed to be visible to B, then A *happen-before* B. This relation is, of course, transitive.

In a single-thread context, this is implicit: if an instruction A precedes an instruction B (A is *sequence-before* B), then A *happen-before* B.

The *inter-thread happen-before* relation is defined between operations A and B when A, executed by one thread, *happen-before* B, executed by another thread. Referring back to the example (Figure 1.3), what is desired is precisely an *inter-thread happen-before* relation between the write to the variable `ready` performed by Thread A and the read of the same variable performed by Thread B. Thanks to the transitive property, this ensures that the read of `i` performed after reading `ready` observes a consistent value. This is achieved by using appropriate `memory_order` tags on the variable `ready`, which must therefore be declared as atomic. The relation that occurs when performing a `store` on an atomic variable with the tag `memory_order_release` and a subsequent load on the same variable with the tag `memory_order_acquire` is defined as `synchronize-with`. It implies that all operations preceding the `store` in the writing thread are visible to all operations following the `load()` (that observes the modification made by the `store`) in the reading thread, thus enforcing an *inter-thread happen-before* relation.

The correct version of the example is shown in Figure 1.4:

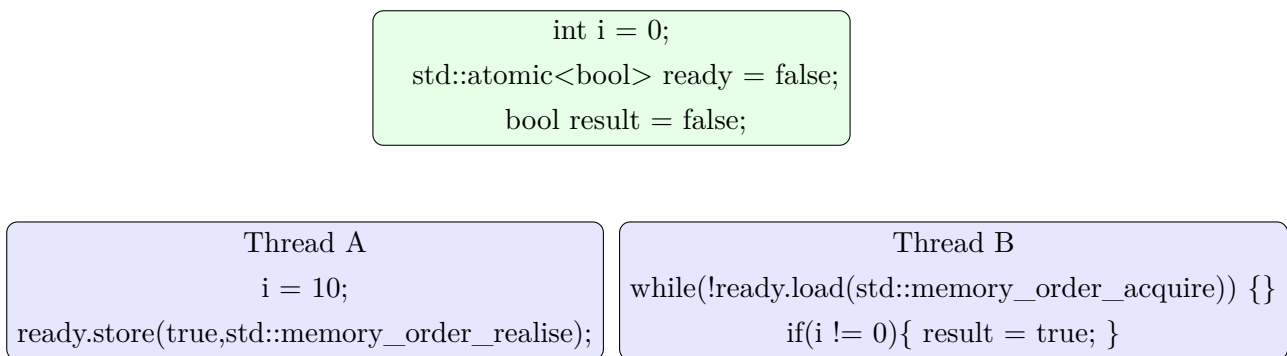


Figure 1.4: Scheme of the properly working example.

`std::memory_order`

Memory orders constrain instruction reordering and, for this reason, their use limits the potential for concurrency and can reduce performance. The purpose and behavior of the `memory_order_acquire` and `memory_order_release` modes have already been discussed, we now present the two memory orders that are respectively less and more restrictive: `relaxed` and `sequential_consistency`. The `std::memory_order_relaxed` mode does not impose any reordering constraints and is therefore the ideal choice for operations on variables that must be atomic but do not require coordination between threads. The `std::memory_order_seq_cst` mode, on the other hand, is the most restrictive and is the default choice when no explicit memory order is specified. It enforces a single global ordering for all atomic operations that use it (see [5]).

1.2.2. Thread-safe data structures

Once the main tools for safe concurrent programming in C++ have been introduced, it is possible to define some concepts that highlight the strengths and weaknesses of using such mechanisms. We begin with the definition of *thread-safe*. A data structure is defined as thread-safe, as described in Chapter 6.1 of [14], if multiple threads can operate concurrently on the structure without causing problematic race conditions. As can be seen from Figure 1.5, thread-safe structures can be divided into two main categories: *blocking* and *non-blocking*. Blocking structures, also referred to as *lock-based*, are those that ensure safe concurrent access by employing synchronization mechanisms such as `mutex`, `condition_variable`, or `future`, which may suspend the execution of a thread in order to coordinate access to shared resources. This type of structure can easily lead to *starvation*, meaning a situation in which a thread is continuously denied access to a lock because it is already held by another thread. The best strategy to mitigate this penalty in blocking structures is to minimize the critical section, thereby reducing the time each thread holds the mutex lock.

Non-blocking structures, on the other hand, rely on the synchronization mechanisms provided by atomic variables and memory orders. Within non-blocking structures, a further distinction can be made: *lock-free* structures allow multiple threads to operate concurrently while ensuring that each thread can still complete its own operation independently of the others. If, moreover, it is guaranteed that every operation completes within a finite number of steps regardless of the behavior of other threads, the structure is said to be *wait-free*. Finally, structures employing so-called *spin locks* are also considered non-blocking. These mechanisms implement active waiting: in this case, instead of being suspended as with a mutex, the thread remains active, repeatedly checking whether the lock has become available. In this sense, spin locks mimic the behavior of mutexes, but without suspending the thread (an example will be shown in paragraph 2.2.4).

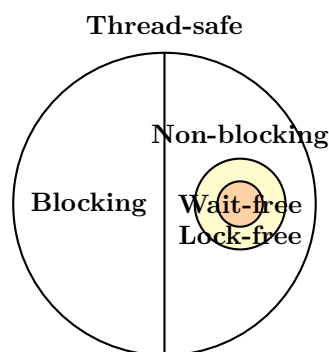


Figure 1.5: Classification of *thread-safe* data structures.

2 | Thread pool implementation

2.1. Introduction

A thread pool, at its simplest, can be seen as a collection of threads that are available to execute tasks [14]. Since it is impossible to have a machine that has a thread available for each concurrent task we want to run, thread pools are used to organize the work between the available threads to speed up the execution with respect to sequential execution of the code.

A thread pool consists of a fixed number of worker threads, defined during instantiation, which usually corresponds to the maximum number of available threads in the system (Obtainable with `std::thread::hardware_concurrency()`).

The pieces of code that are meant to be ran by the threads, called tasks or jobs, are stored in appropriate thread-safe containers, accessed like double-ended queues. There are two ways the containers can be implemented inside the thread pool: there could be a single dequeue shared between all the threads, or a queue for each thread. Each of the two possibilities comes with its benefits and drawbacks:

- A single dequeue is easier to implement and allows simpler distribution of the work between threads, but care must be taken in the definition of the queue to ensure that it is thread-safe. Furthermore, as the number of threads increases, so do the accesses to the data structure and the conflicts between threads, slowing down the execution.
- Dedicated dequeues for each thread can allow the thread pool to be entirely lock free, provided that there is no need to wait for jobs to be completed, greatly simplifying the code. Furthermore, the presence of dedicated dequeues reduces the possibility of conflict between threads, making the code faster compared to the previous solution as the number of threads increases. The main drawback is that it is important to correctly distribute the jobs between the threads: job stealing can be introduced to allow free threads to take tasks from other threads, but in this case it is necessary

for the structure to be thread-safe.

With dedicated dequeues for each thread, it is important to efficiently distribute the workload between all the available workers. Because of this, we introduce two concepts:

- Scheduling: task assignment heuristic in the thread pool, which sends incoming tasks to a worker, depending on the selected strategy.
- Stealing: dynamic load balancing activated after the initial scheduling, which allows idle threads to steal tasks from other workers. This is useful in case load imbalance appears due to different job workloads between the threads. It is important to note that the load balancer is separate from the scheduler, as their objectives are different.

The scheduler is not responsible for reaching perfect load balancing, which is responsibility of the stealing strategy: load balancing is something which must be reached only dynamically, during the execution of the thread pool, and cannot be determined statically when a job arrives, as at this point we do not have any guess of how much time its execution will take). Its job is to prevent initial imbalances, such as the case in which all the jobs are sent to one thread: it should not point to perfect balancing, which is redundant and costly. A scheduler should place the jobs so as to prevent contention, that is to avoid sending all the jobs only to a few workers (otherwise lock contention due to stealing on their queues will be a problem), while attempting a fair and cheap balancing. In fact, the scheduler should not compete with the load balancer.

The worker threads of the thread pool are instantiated with a worker loop function, that performs the following instructions:

- Registers the worker in the map that contains all the workers of the thread pool.
- Tries to pop a job from the worker's queue.
- If the pop is not successful, tries to steal a job from another queue, following the selected stealing policy.
- If stealing is not successful, puts the worker to sleep until a new job is inserted in the queue.

This worker loop function executes until it is ordered to stop by the program.

2.2. `synchro_queue`

The C++ *Standard Template Library* does not provide containers specifically designed for safe operations in a multi-threaded environment. For this reason, in order to use them later within the thread pool as the queues of the individual workers, we implemented a thread-safe blocking double-ended queue designed for multi-producer, multi-consumer scenarios. As mentioned in Section 1.5, a thread-safe blocking structure makes use of a `mutex` to protect the critical section of the code, thus serializing concurrent accesses. The goal was therefore to minimize the size of the critical section when operating on the queue, in order to achieve maximum efficiency. As highlighted in Chapter 6.1 of [14], the smaller the protected region, the fewer operations are serialized, and the greater the potential for concurrency.

2.2.1. `synchro_queue` template class

The `synchro_queue<value_type, access_model>` data structure is implemented as a ring buffer built on top of a `std::vector<elem>`, as illustrated in Figure 2.1, that ensures efficiency thanks to the contiguous memory layout of its elements, and allows thread-safe concurrent operations on different parts of the queue, since no memory reallocations are required. Each slot of the buffer is represented by an `elem` structure, which stores the actual data and a state flag indicating whether the cell is currently occupied or empty. The data field is implemented as a `std::optional<value_type>`, containing either a valid element or `std::nullopt`, if the corresponding cell is empty.

Two integer indices, `head_` and `tail_`, define the logical boundaries of the queue:

- `head_` points to the first occupied element, i.e. the logical front of the queue;
- `tail_` points to one position past the last occupied element, following the classic circular-buffer convention.

During push and pop operations, the private mutex of the `synchro_queue` is first acquired to update `head_` and `tail_` consistently, effectively reserving the position where the operation will take place. Once the indices have been updated, the mutex is released and the actual modification of the corresponding `elem` is performed outside the critical section.

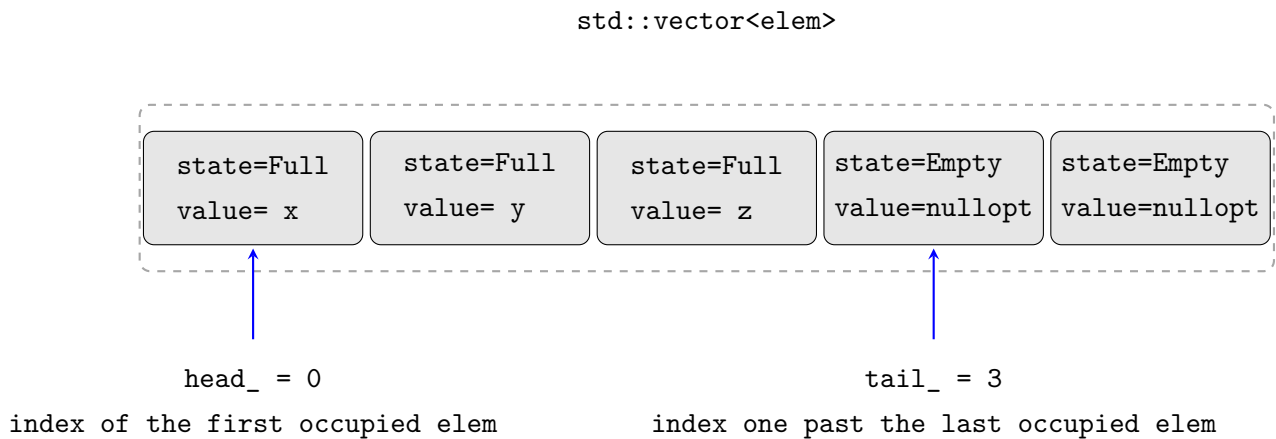


Figure 2.1: Internal layout of `synchro_queue`.

Note: By convention, the `head_` and `tail_` indices coincide when the queue is either full or empty.

Template parameters The `synchro_queue` class is defined as a template parameterized by two types: the data type stored in the queue and the access model used to regulate its synchronization behavior:

```
template<typename value_type, typename access_model>
class synchro_queue;
```

- `value_type` denotes the type of the elements stored in the queue.
- `access_model` selects the synchronization strategy.

Three partial specializations of `synchro_queue` with respect to the `access_model` parameter are provided:

```
struct relaxed {};
struct deferred {};
struct blocking {};
```

Each specialization provides a different semantic and performance profile, and defines its own internal `elem` structure:

- **relaxed:** the most lightweight and efficient variant, providing weaker semantics. Operations may fail and return a negative result if the queue is full or empty, or if contention prevents immediate completion.
- **deferred:** provides stronger semantics at the cost of higher memory usage and overhead. If an operation cannot complete immediately, the calling thread is sus-

pended and resumed when it can proceed. Operations may still fail if the queue is full or empty.

- **blocking**: provides the strongest semantics but exhibits the worst performance due to thread suspension and synchronization overhead. Operations are guaranteed to complete, as threads are blocked until the requested operation can be successfully performed.

2.2.2. Common API

In this paragraph we present the general API of `synchro_queue`.

`Synchro_queue()`

Default constructor.

`Synchro_queue(int n)`

Returns a queue with `n` empty elements.

Note: The `elem` structure is default-constructible.

`template <typename Iterator>`

`synchro_queue(Iterator begin, Iterator end)`

initializes the `Synchro_queue` from a container of type vector, array, or list with elements ranging from `begin` to `end` iterator

`int get_head() const`

Returns the value of `head_` without acquiring `synchro_queue`'s mutex.

`int get_tail() const`

Returns the value of `tail_` without acquiring `synchro_queue`'s mutex.

`int get_size() const`

Returns the value of `size_` without acquiring `synchro_queue`'s mutex.

`void clear()`

Empties the queue, resetting the indices `head_` and `tail_` to 0.

`void resize(int n)`

Clears the queue and resizes it to a new capacity `n`.

```
bool push_front(value_type val)
```

Attempts to insert a copy of `val` into the element preceding the one pointed to by `head_`. Returns `true` if the operation succeeds, `false` otherwise.

Note: The `synchro_queue` stores a copy of the value passed to `push_front`, as is good practice in multithreaded programming, because this avoids imposing constraints on the lifetime of the input object.

```
bool push_back(value_type val)
```

Attempts to insert a copy of `val` into the element pointed to by `tail_`. Returns `true` if the operation succeeds, `false` otherwise.

```
std::optional<value_type> pop_front()
```

Returns a `std::optional` containing the element pointed to by `head_`. If the queue is empty, the optional contains `std::nullopt`.

```
std::optional<value_type> pop_back()
```

Returns a `std::optional` containing the element preceding the one pointed to by `tail_`. If the queue is empty, the optional contains `std::nullopt`.

We now move on to the analysis of the individual `access_model` variants.

2.2.3. `access_model::relaxed`

Semantics

This version of `Synchro_queue` adopts weaker semantics: a `push` or `pop` operation can fail (returning `false` or `std::nullopt`) even when the queue is neither full nor empty. In particular, once the queue's mutex has been acquired, the operations behave in a non-blocking manner. The thread then checks whether the target element is immediately available, i.e., not currently being accessed by any other thread. If the element is unavailable, the operation aborts and returns a negative result. Element availability is determined based on its state.

Each element of the queue consists of an optional value and an atomic state:

- Empty if the value store is `std::nullopt` and no other thread is operating on that `elem`.

- Busy if other thread is operating on that `elem`.
- Full if the value store is not `std::nullopt` and no other thread is operating on that `elem`.

These states are defined via static constants.

```
static constexpr int Full = 0;
static constexpr int Empty = 1;
static constexpr int Busy = 2;
struct elem{
    std::atomic<int> state_ = Empty;
    std::optional<value_type> v_;
};
```

Explanation of the push and pop methods

Each push or pop operation is divided into two phases:

1. First phase: the queue's mutex is acquired to update the `head_` or `tail_` index, depending on whether the operation concerns the head (*front*) or the tail (*back*). In this phase, it is checked whether insertion or removal is possible. The operation is admissible if the state of the element to be operated on is **Empty** for a push or **Full** for a pop. The index is updated as if the action had already been completed, and the corresponding elements state is set to **Busy**.
Note: checking whether the queue is full or empty is not necessary, since this is implicit in checking the state of the element to be operated on.
2. Second phase: after releasing the mutex, the actual operation takes place. The element's state is updated from **Busy** to **Empty** after a pop operation and from **Busy** to **Full** after a push operation.

By doing so, the relaxed access model uses a weaker semantic: if a method attempts to operate on an `elem` while another thread is already performing an operation on the same `elem`, it detects the elements **Busy** state, and does not wait for the previous operation to complete. Such waiting, as show in Section 2.2.4, is instead implemented in the **deferred** version, thereby strengthening the semantics at the expense of efficiency.

Syntax of the push and pop methods

bool push_front(value_type val)

Attempts to insert a copy of **val** into the element preceding the one pointed to by **head_**. If the element's state is not **Empty**, the operation fails and returns **false**; otherwise, it succeeds and returns **true**.

bool push_back(value_type val)

Attempts to insert a copy of **val** into the element pointed to by **tail_**. If the element's state is not **Empty**, the operation fails and returns **false**; otherwise, it succeeds and returns **true**.

std::optional<value_type> pop_front()

Attempts to remove the element pointed to by **head_**. If the element's state is not **Full**, the operation fails and returns **std::nullopt**; otherwise, it returns a **std::optional** containing the element's value.

std::optional<value_type> pop_back()

Attempts to remove the element preceding the one pointed to by **tail_**. If the element's state is not **Full**, the operation fails and returns **std::nullopt**; otherwise, it returns a **std::optional** containing the element's value.

We now examine the implementation of the **front** methods only. For brevity, we do not report the implementation of the **push_back** and **pop_back** methods, as they follow a similar reasoning.

```

1  bool push_front(value_type val) {
2      std::unique_lock<std::mutex> loc(m_);
3      int new_head = (head_ == 0) ? (size_ - 1) : (head_ - 1);
4      if (queue_[new_head].state_.load(std::memory_order_acquire) != Empty)
5          return false;
6      queue_[new_head].state_.store(Busy, std::memory_order_relaxed);
7      head_ = new_head;
8      loc.unlock();
9  }
```



```

10     queue_[new_head].v_ = std::move(val);
11     queue_[new_head].state_.store(Full, std::memory_order_release);
12     return true;
13 }

```

The operation starts by acquiring a mutex to ensure exclusive access to the queue indices. The index where the push is performed is computed. The availability of the target `elem` is checked using a `load` with `memory_order_acquire`, which synchronizes with the corresponding `release` performed by a previous `pop`. Once the cell is found to be available, its state is set to `Busy` using `memory_order_relaxed`, as no additional synchronization is required. The mutex is then released. Finally, the value is inserted and the `elem` state is updated to `Full` using `memory_order_release`, synchronizing with a future `load(acquire)` performed by a `pop` operation and ensuring visibility of the insertion.

```

1  std::optional<value_type> pop_front() {
2      std::unique_lock<std::mutex> loc(m_);
3      int h = head_;
4      if (queue_[h].state_.load(std::memory_order_acquire) != Full){
5          return std::nullopt;}
6      queue_[h].state_.store(Busy, std::memory_order_relaxed);
7      head_ = (head_ == size_ - 1) ? (0) : (head_ + 1);
8      loc.unlock();
9      // pop
10     value_type ret = std::move(queue_[h].v_.value());
11     queue_[h].v_ = std::nullopt;
12     queue_[h].state_.store(Empty, std::memory_order_release);
13     return ret;
14 }

```

The `pop` operation follows the same synchronization pattern as `push`, with the only difference that a cell is considered available when its state is `Full` rather than `Empty`.

Note: It is not necessary to use an atomic *compare-exchange* operation, as described in Section 1.2.1, to change the state from `Empty` to `Busy` or from `Full` to `Busy` after the `if` statement. This is because, outside the mutex, the state can only transition from `Busy` to another value. Therefore, once the `Empty` or `Full` state is observed during the mutex lock, it is guaranteed not to be modified concurrently, and no atomic read-modify-write operation is needed.

empty() method

The `empty` method within the thread pool is used to determine whether there are still jobs to be executed in the queues. For this reason, it must provide a reliable reading of the queue's state.

bool `empty()` **const**

returns `true` if the queue is empty, `false` otherwise.

```

1  bool empty() const {
2      std::lock_guard<std::mutex> loc(m_);
3      if(head_ == tail_){
4          while(true){
5              if(queue_[tail_].state_.load(std::memory_order_acquire)==Empty){
6                  return true;}
7              if(queue_[tail_].state_.load(std::memory_order_acquire)==Full){
8                  return false;}
9          }
10     }else{return false;}}

```

Aacquiring the mutex to prevent modifications during execution.

This condition is true when the queue is either empty or full.

Checking the state of the elements pointed to by `head_` and `tail_` allows distinguishing whether the queue is empty or full.

The `while` loop allows waiting for any ongoing operation to complete and then change the state from `Busy` to either `Full` or `Empty`.

The guarantee that the queue is actually empty, that is, that the effect of the actual last pop is visible when the thread performs the `load(acquire)` reading the `Empty` state, is provided by the synchronizes-with relationship between this `load(acquire)` and the preceding `store(release)` performed by the last pop operation.

2.2.4. access_model::deferred

Semantics

Under `access_model::deferred` a pop or push operation fails only when the queue is empty or full, respectively. A synchronization mechanism is therefore provided for concurrent operations on the same element of the queue.

It is not guaranteed that the final order of a concurrent operation on the same element

will be preserved, as highlighted in the example below.

Let us now analyze `access_model::deferred`, focusing on the `elem` structure it defines and how it synchronizes operations.

```
static constexpr int Empty = 1;
static constexpr int Full = 0;
struct elem{
    int state_ = Empty;
    std::optional<value_type> v_;
    mutable std::mutex m_el_;
    std::condition_variable cv_ready_to_push_;
    std::condition_variable cv_ready_to_pop_;
    int count_pop_ = 0;
};
```

The state of the element now has a binary form: `Empty` or `Full`. For consistency with the `relaxed` version, static constants are also used here.

Each element is also associated with a dedicated mutex, which guarantees exclusive access during push and pop operations, and a pair of condition variables used to correctly coordinate insertions and removals.

Finally, a counter of pop operations has been introduced, used by the `empty()` method to reliably determine the state of the queue.

Explanation of the push and pop methods

Each push or pop operation is divided into two phases:

1. First phase: the queue's mutex is acquired to update the (`head_` or `tail_`) index. It is only checked that the queue is not empty or full (for pop or push). The index is updated as if the action had already been completed and the integer value representing the actual position to operate on is returned.
2. Second phase: after releasing the queue's mutex, the element's mutex is acquired. The thread then checks whether the operation can proceed. If it can, the operation is executed; otherwise, the thread waits on the condition variable until it is notified. Once the operation is complete, the thread notifies any other waiting threads.

We now show an example for clarity:

Let T1, T2, T3 be three different threads operating concurrently on a shared `synchro_queue`, as shown in Figure 2.2. T1 and T3 want to push two elements at the front of the queue,

while T2 wants to pop one element from the front of the queue. As can be seen by the position of the head and tail pointers, the queue is partially occupied in its initial state.

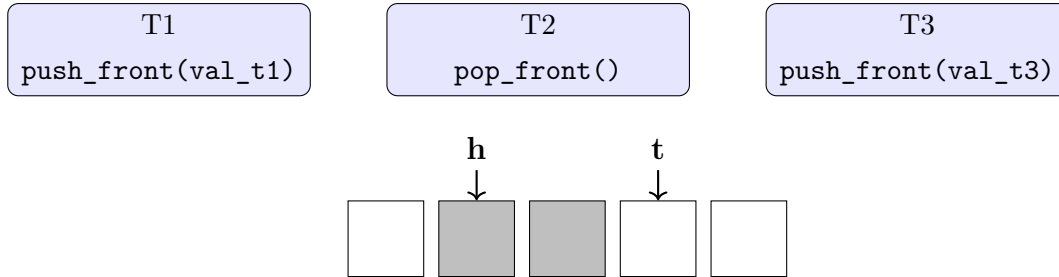


Figure 2.2: Initial condition of the queue and functions associated with the threads.

Assume that mutex acquisition enforces an interleaving where the `pop` alternates with the two `push` operations, so that all three operations act on the element at position 0. Figure 2.3 shows the state of the queue after all threads have completed phase 1.

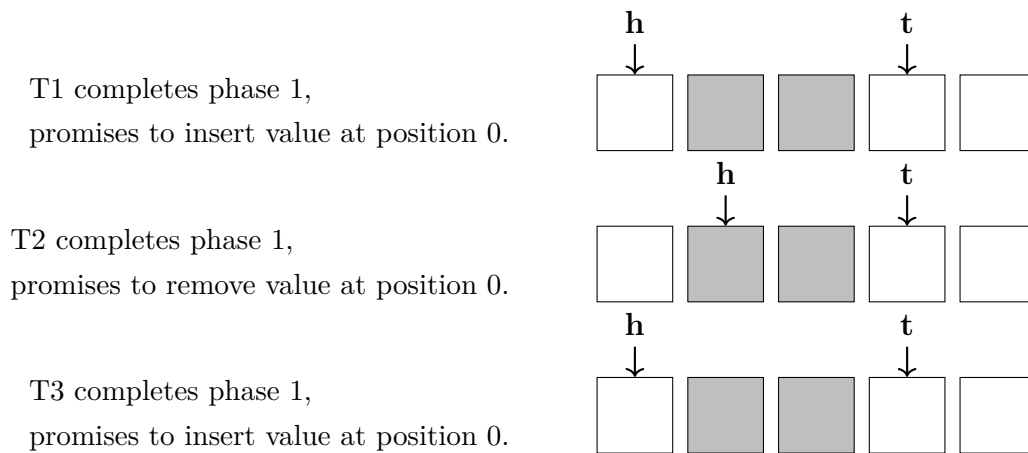


Figure 2.3: Execution of phase 1 in the example.

All threads now attempt to acquire the mutex of the cell at position 0 in order to perform the actual operation. The order in which threads acquire the element-level mutex is not guaranteed to match the order in which they acquired the queue-level mutex. However, thanks to the use of condition variables, once a thread locks the element mutex, it is allowed to proceed only if it is effectively its turn to operate.

As a consequence, it is guaranteed that one of the two `push` operations performed by threads T1 or T3 completes Phase 2 and inserts a value into `elem0` before the `pop` operation of thread T2 is executed. Nevertheless, it is not possible to determine in advance the value returned by `pop_front()`, as this depends on which thread performs the `push`

operation before T2 executes the `pop`. Figure 2.4 illustrates one possible outcome for this example.

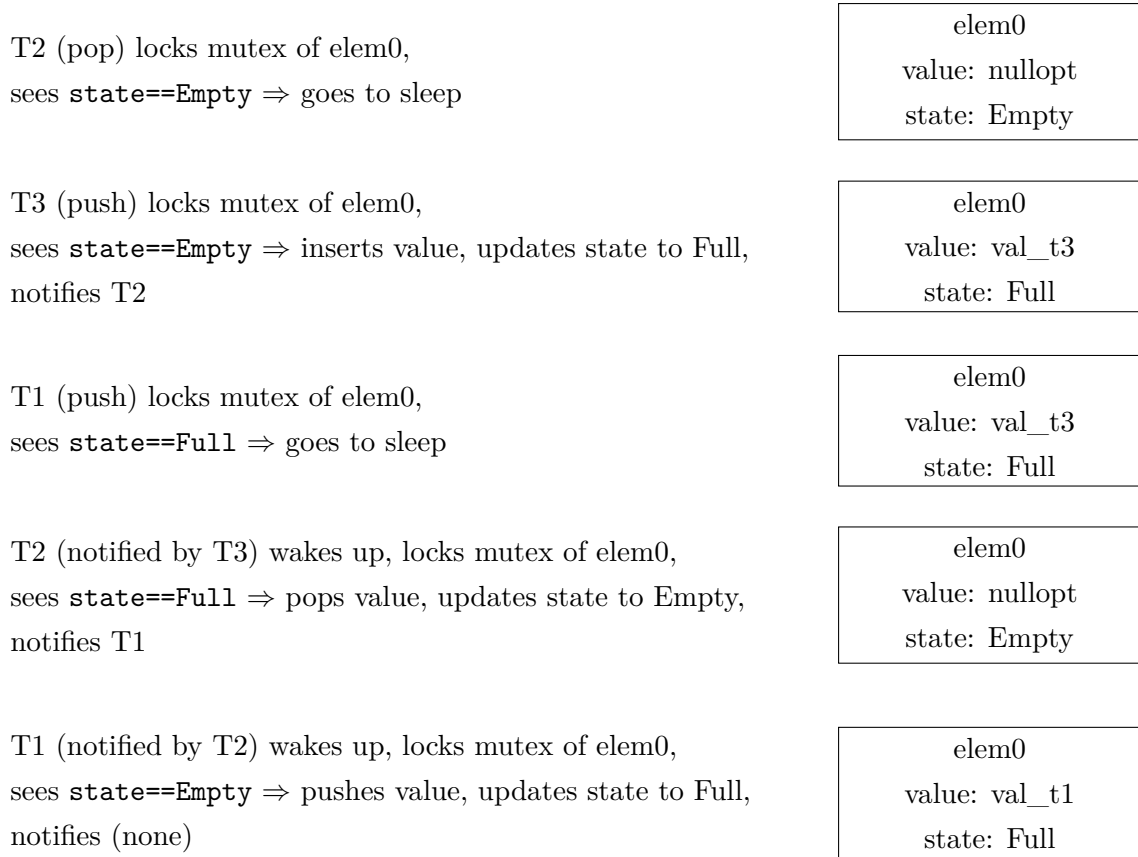


Figure 2.4: Execution of phase 2 of the example.

In the example shown, the final state is such that `elem0.value = val_t1` and thread T2 pops `val_t3`. However, a priori, the alternative outcome in which `elem0.value = val_t3` and T2 pops `val_t1` is equally valid and would have occurred if thread T1 had acquired the mutex of `elem0` before T3.

Why introduce `empty_queue_`? Under `access_model::deferred`, unlike with `relaxed`, the state of an element cannot be used as a discriminator to determine whether `head_ == tail_` corresponds to an empty or a full queue. This is because, in the `deferred` model, the transition from `Empty` to `Full` (or vice versa) may occur outside the mutex, a sequence that is not possible in the `relaxed` specialization, where only the transition from `Busy` to `Full/Empty` is allowed outside the critical section. The execution scenario presented below illustrates how relying on the state of the elements pointed to by `head_` and `tail_` may lead to an incorrect classification of the queue state.

Example Potential issue in full queue scenario using `head_`, `tail_`

Initial state:	Queue is Full. <code>head</code> and <code>tail</code> point to the same index.
Step 1:	A <code>pop</code> and a <code>push</code> start concurrently. Both complete only their first phase, moving the <code>head_</code> index forward and backward, so it still coincides with <code>tail_</code> .
Step 2:	The first <code>pop</code> locks the elements mutex, removes the <code>value_type</code> , and sets the state to <code>Empty</code> .
Step 3:	A second <code>pop</code> starts (phase one). It sees <code>head == tail</code> but the elements state is <code>Empty</code> (because the first <code>pop</code> has already set it to <code>Empty</code>).
Step 4:	The second <code>pop</code> incorrectly assumes the queue is empty (<code>head == tail</code> and element state <code>Empty</code>) and returns instead of waiting for the pending <code>push</code> .

For this reason, we introduce an additional member variable: `bool empty_queue_`, which is true when the indices `head_` and `tail_` coincide because the queue is empty, false otherwise. It is used to discriminate, during the first phase, when `head_` and `tail_` coincide, whether the queue is full or empty, and it is reliable because it is modified only during the first phase of the methods while the queue mutex is locked.

Syntax of the `push` and `pop` methods

```
bool push_front(value_type val)
```

Attempts to insert a copy of `val` into the element preceding the one pointed to by `head_`. If the queue is full, the operation fails and returns `false`; otherwise, it succeeds and returns `true`.

```
bool push_back(value_type val)
```

Attempts to insert a copy of `val` into the element pointed to by `tail_`. If the queue is full, the operation fails and returns `false`; otherwise, it succeeds and returns `true`.

```
std::optional<value_type> pop_front()
```

Attempts to remove the element pointed to by `head_`. If the queue is empty, the operation fails and returns `std::nullopt`; otherwise, it returns a `std::optional` containing the element's value.

```
std::optional<value_type> pop_back()
```

Attempts to remove the element preceding the one pointed to by `tail_`. If the queue is empty, the operation fails and returns `std::nullopt`; otherwise, it returns a `std::optional` containing the element's value.

We now show the implementation of the `push_front` and `pop_front` methods. For brevity, we do not report the implementation of the `push_back` and `pop_back` methods, as they follow the same reasoning.

```

1  bool push_front(value_type val) {
2      std::unique_lock<std::mutex> loc(m_);
3      if (head_ == tail_ && !empty_queue_) { return false; } ←
4      empty_queue_ = false;
5      int new_head = (head_ == 0) ? (size_ - 1) : (head_ - 1);
6      head_ = new_head;
7      loc.unlock();
8
9      std::unique_lock<std::mutex> loc_el(queue_[new_head].m_el_);
10     queue_[new_head].cv_ready_to_push_.wait(
11         loc_el, [this, new_head]() { return queue_[new_head].state_ == Empty; });
12     queue_[new_head].v_ = std::move(val);
13     queue_[new_head].state_ = Full;
14     loc_el.unlock();
15     queue_[new_head].cv_ready_to_pop_.notify_one();
16     return true;
17 }
```

Uses `empty_queue_` to distinguish whether `head_ == tail_` means the queue is full or empty

The operation starts by acquiring a mutex to ensure exclusive access to the queue indices. If the queue is full, the method returns immediately; otherwise, the head index is updated to reserve space for the new element. The mutex is then released to reduce contention.

The target cell is locked and the thread waits on `cv_ready_to_push_` until the cell state becomes `Empty`. Once the value is written, the cell state is set to `Full`, the element-level lock is released, and any `pop` operation waiting on that element is notified.

```

1  std::optional<value_type> pop_front() {
2      std::unique_lock<std::mutex> loc(m_);
3      if (empty_queue_) { return std::nullopt; }
4      int h = head_;
5      head_ = (head_ == size_ - 1) ? (0) : (head_ + 1);
6      if (head_ == tail_) { empty_queue_ = true; }
7      queue_[h].count_pop++; ← Increment the pop counter for the element, to
8      loc.unlock();           signal that a pop still has to be executed.
9
10     std::unique_lock<std::mutex> loc_el(queue_[h].m_el_);
11     queue_[h].cv_ready_to_pop_.wait(loc_el, [this, h]() {return queue_[h].state_ == Full;});
12     value_type ret = std::move(queue_[h].v_.value());
13     queue_[h].v_ = std::nullopt;
14     queue_[h].state_ = Empty;
15     queue_[h].count_pop--; ← Decrements the pop counter of the element to
16     loc_el.unlock();       signal that the pop has been executed.
17     queue_[h].cv_ready_to_push_.notify_one();
18     return ret;
19 }

```

The behavior of `pop` is analogous to that of `push`, with the addition of a per-element `count_pop_` update.

Note: The correct visibility of the actual operations performed at the end of the methods is ensured by the synchronize-with relationship that occurs between unlocking and subsequently locking the element's mutex, as explained in [6]. This is why, differently from the `relaxed` version, the element's state does not need to be atomic.

About the necessity of 2 condition variables As illustrated in the execution example below, relying on a single condition variable may lead to deadlock in situations where a `pop` notifies another `pop`, or a `push` notifies another `push`.

Example Potential deadlock scenario with 1 CV

Initial state:	A sequence of operations arrives: <code>push_1</code> , <code>pop_1</code> , <code>push_2</code> , <code>pop_2</code> . Indices move, targeting the same empty cell.
Step 1:	Both <code>pop_1</code> and <code>pop_2</code> acquire sequentially the element's mutex first. They find the cell empty and go to sleep waiting for a notification.
Step 2:	<code>push_1</code> locks the mutex, inserts the element, and then notifies one sleeping thread (<code>pop_2</code>).
Step 3:	Meanwhile, <code>push_2</code> locks the mutex before <code>pop_2</code> wakes up, sees the cell is full, and also goes to sleep.
Step 4:	The notified <code>pop_2</code> wakes up, performs the pop operation, and incorrectly notifies <code>pop_1</code> instead of the waiting <code>push_2</code> . As a result <code>push_2</code> and <code>pop_1</code> both remain stuck waiting on the condition variable, leading to a deadlock.

Alternative synchronizations We point out that different synchronization alternatives are possible:

- **A single condition variable with `notify_all()`**

This guarantees that, after a `pop`, any waiting `push` is also notified (or vice versa). However, this approach would lead to a greater number of spurious wake-ups and thus a loss of efficiency.

- **Spin-lock**

This strategy was introduced in Section 1.2.2. The use of condition variable could be replaced by a double while loop and two flags indicating that a push or a pop is currently executing, ensuring that two identical methods do not follow one another.

We show an example of how the `elem` struct would be modified and of the implementation of the `push_front` method. The `elem` defined in the deferred specialization becomes:

```

1 struct elem{
2     std::atomic<bool> empty_;
3     std::optional<value_type> v_;
4     std::atomic<bool> flag_push_;
5     std::atomic<bool> flag_pop_;
6 };

```

It is made atomic, as in the relaxed version, to guarantee the correct visibility of the actual operation through the synchronize-with relationship provided by `store(release)` `load(acquire)`

true while push

true while pop

In `push_front`, instead of locking the elements mutex and calling `wait()`, a double while loop is used:

```

1  /* lock queue mutex, find index h where to insert the element, unlock queue mutex */
2  bool adding = false;
3  while(
4      !queue_[h].flag_push_.compare_exchange_weak(adding, true, std::memory_order_acquire)
5      ){adding = false;}
6  while(!queue_[h].empty_.load( std::memory_order_acquire)){
7      /* actual push */

```

The second ensures that the push waits for any previous pop to empty the element, and then proceeds with the actual insertion.

In the first while, an atomic read-and-modify of `flag_push_` is performed so that only one push at a time can pass this step

This "busy wait", although it can reduce the execution time of the method, leads to higher CPU usage, which overall makes this strategy less efficient.

empty() method

The `empty()` method, as in the `relaxed` case, must provide a reliable view of the queue's state.

```
bool empty() const
```

returns `true` if it is empty and `false` otherwise.

```

1  bool empty() const {
2      std::lock_guard<std::mutex> loc(m_);
3      if(empty_queue_){
4          bool not_empty = true;
5          while(not_empty){
6              std::unique_lock<std::mutex> loc_el(queue_[tail_].m_el_);
7              not_empty = (queue_[tail_].count_pop_ != 0);
8              loc_el.unlock();
9              std::this_thread::yield();
10         };
11         return true;
12     }
13     else
14         return false;
15 }

```

Locking the queue mutex guarantees no other pop/push starts phase 1 and allows a consistent read of `empty_queue_`.

While allowing to wait for `count_pop_` to reach zero, meaning the actual operations have completed (recall: `count_pop_` is incremented by pop in phase 1 and decremented at the end of phase 2).

If true, the queue is empty, but any ongoing push or pop completing phase 2 must be awaited.

Note: the guarantee that reading the value `count_pop == 0`, and hence that `empty()` returns `True`, is consistent with the actual operation on the cell is provided by the synchronize-with relation between the previous unlock and subsequent lock of `elem`'s mutex [6].

An alternative would have been to make the counters atomic, explicitly applying acquire and release memory orders during read and modify operations, as is the case in the relaxed version for managing the elements state. However, this choice would have slowed down pop operations, since modifying atomic variables is more expensive than modifying ordinary variables.

It was therefore preferred to keep a non-atomic counter and read it while holding the mutex lock, allowing a performance degradation of the `empty()` method, which favors reliability over efficiency, in exchange for faster pop operations.

Note: counting only the pop operations is sufficient because `empty_queue == True` only if the number of pushes corresponds to the number of pops performed.

2.2.5. `access_model::blocking`

Semantic

Under the `access_model::blocking`, a normal pop or push operation fails only when the queue is empty or full, respectively. A `*_wait` operation, on the other hand, can only return a successful result. Therefore, if it cannot be completed (e.g., `push_wait()` on a

full queue), it suspends the executing thread on a condition variable until it is notified that the operation can proceed (e.g., a `pop` on the full `synchro_queue`). It is not guaranteed that the final order of a concurrent operation on the same element will be preserved.

This specialization has the same structure of `elem` found in `access_model::deferred` and identical push and pop methods. In addition, it provides push and pop methods that only complete their execution when the operation can succeed; until then, if necessary, they suspend the thread and wait on a condition variable. Therefore, this requires the addition of two private data members:

```
std::condition_variable cv_can_pop_;
std::condition_variable cv_can_push_;
```

The consequent notification instructions inside every pop and push have to be introduced as well, which leads to a degradation in performance and hence the choice to make this strategy separate from `access_model::deferred`.

push_wait and pop_wait methods

For brevity, only the additional `*_front_or_wait` methods are presented:

```
bool push_front_or_wait(value_type val)
```

Insert a copy of `val` into the element preceding the one pointed to by `head_` and returns `true`. If the queue is full, the calling thread is suspended until it can proceed.

```
bool push_front_or_wait_for(value_type val, int s)
```

Attempts to insert a copy of `val` into the element preceding the one pointed to by `head_` and returns `true`. If the queue is full, the calling thread is suspended for at most `s` seconds. If it becomes possible to proceed before the timeout expires, the operation completes; otherwise, fails and returns `false`.

```
value_type pop_front_or_wait()
```

Removes and returns the element pointed to by `head_`. If the queue is empty, the calling thread is suspended until it can proceed.

```
std::optional<value_type> pop_front_or_wait_for(int s)
```

Attempts to removes and returns the element pointed to by `head_`. If the queue is empty, the calling thread is suspended for at most `s` seconds. If it becomes possible to proceed before the timeout expires, the operation completes; otherwise, fails and returns `std::nullopt`.

We now present the implementation of the `push_front_or_wait` and `push_front_or_wait_for` methods. All `push_front_*` methods internally rely on the private member function `push_front_` to avoid code duplication. The same pattern is used for the `push_back_*`, `pop_front_*`, and `pop_back_*` methods.

```

1  //private helper method
2  bool push_front_(value_type& val, std::unique_lock<std::mutex>& loc) {
3      empty_queue_ = false;
4      int new_head = (head_ == 0) ? (size_ - 1) : (head_ - 1);
5      head_ = new_head;
6      loc.unlock();
7      cv_can_pop_.notify_one();    // notify any waiting pop_*_or_wait*
8      std::unique_lock<std::mutex> loc_el(queue_[new_head].m_el_);
9      queue_[new_head].cv_ready_to_push_.wait(
10         loc_el, [this, new_head]() { return queue_[new_head].state_ == Empty; });
11     queue_[new_head].v_ = std::move(val);
12     queue_[new_head].state_ = Full;
13     loc_el.unlock();
14     queue_[new_head].cv_ready_to_pop_.notify_one();
15     return true;
16 }
17
18 //public
19 bool push_front(value_type val) {
20     std::unique_lock<std::mutex> loc(m_);
21     if (head_ == tail_ && !empty_queue_) {return false;} //if the queue is full abort
22     return push_front_(val, loc);
23 }
```

```

1  bool push_front_or_wait_for(value_type val, int s) {
2      std::unique_lock<std::mutex> loc(m_);
3      bool flag = cv_can_push_.wait_for(loc, std::chrono::seconds(s), [this]() { ←
4          return this->head_ != this->tail_ || this->empty_queue_;
5      }); // wait up to s seconds for the queue to become non-full
6      if (!flag) { return false; } // abort if the timer expires
7      return push_front_(val, loc);
8  }
9
10 bool push_front_or_wait(value_type val) {
11     std::unique_lock<std::mutex> loc(m_);
12     cv_can_push_.wait(loc, [this]() {
13         return this->head_ != this->tail_ || this->empty_queue_;
14     }); // wait until the queue becomes non-full
15     return push_front_(val, loc);
16 }

```

used the `wait_for()` method of the condition variable, which returns `false` if the timeout expires or `true` if the condition is met

2.3. Thread pool

A thread pool is a concurrency abstraction designed to execute a collection of tasks using a set of long-lived threads. Instead of creating and destroying threads for each task, the thread pool manages a group of worker threads that repeatedly try to retrieve and execute tasks from shared or distributed queues. In this context, a *job* denotes an executable unit of work, while a *worker* is a thread responsible for executing such jobs.

The `threadpool` class implemented in this work consists of a set of workers, each associated with a local queue of jobs, implemented as instances of an inner `worker` class defined within the `threadpool` class. From an architectural perspective, a thread pool relies on two distinct mechanisms: *scheduling* and *stealing*. Scheduling defines the policy used to assign newly submitted jobs to workers and determines the initial distribution of work within the pool. Schedulers therefore act as dispatching strategies applied at job submission time. Stealing is a dynamic mechanism applied after the initial scheduling phase and is used to mitigate workload imbalance among workers. Due to heterogeneous job execution times, some workers may become idle while others remain busy. In such cases, stealing enables idle workers to retrieve jobs from the queues of busy ones, thereby redistributing work and improving overall throughput. The implemented `threadpool` adopts this architecture by associating each worker with a private `synchro_queue` (in the *relaxed* version) containing jobs of type `std::function<void()>`. New jobs submitted to the pool are scheduled by inserting them into the queues of selected workers. Each worker

executes jobs from the front of its own queue and, when no local job is available, attempts to steal jobs from the back of the queues of other workers. A schematic representation of our implementation of the thread pool can be found in Figure 2.5.

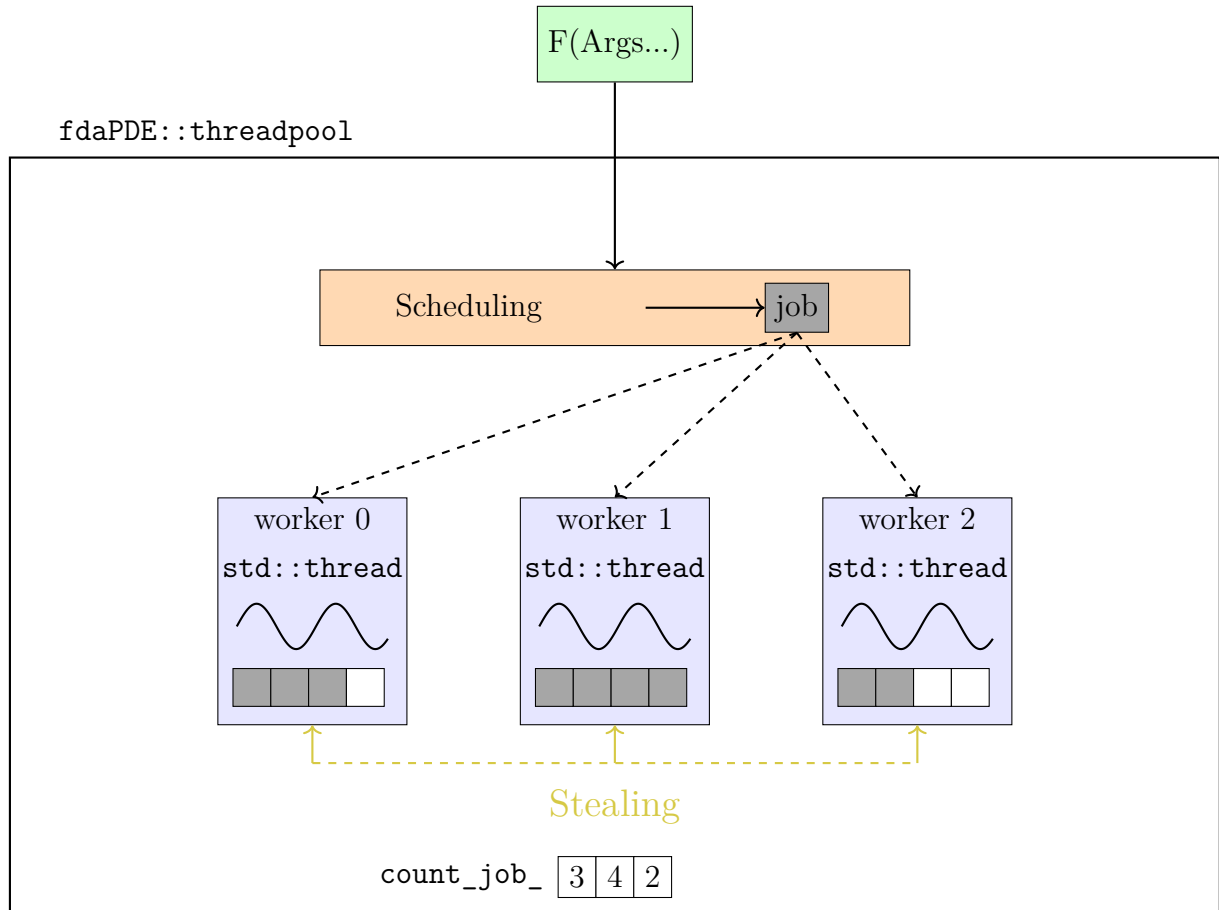


Figure 2.5: Schematic representation of `threadpool`.

2.3.1. worker class

Each `worker` object encapsulates all the information required by a single execution thread within the thread pool. In particular, each worker is identified by an index (`indx_`), which represents its position in the vector of workers managed by the `threadpool`. Each worker owns a private `synchro_queue` used to store pending jobs, together with a dedicated `std::thread` object that executes the worker loop defined by the `threadpool`. The worker thread is initialized at construction time and starts executing the `workerloop()` method of the enclosing `threadpool` instance. Synchronization primitives are also encapsulated within the worker. A mutex and a condition variable are used to suspend the execution of the worker thread when no jobs are available and to resume it upon notification. This mechanism allows workers to block efficiently in the absence of work,

avoiding busy waiting. Finally, the `worker` class provides a set of wrapper methods for basic operations on its internal data members, such as queue access, thread management, and synchronization.

```

1  class worker {
2      private:
3          int indx_;
4          fdapde::synchro_queue<job, fdapde::relaxed> sync_queue_;
5          std::thread t_;
6          bool stop_ = false;
7          mutable std::mutex m_;
8          mutable std::condition_variable cv_;
9      public:
10         friend class threadpool;
11         worker(int n, void (threadpool::*worker_loop)(int), threadpool* Th, int idx) :
12             indx_(idx), sync_queue_(n), t_(worker_loop, Th, idx) {};
13         std::unique_lock<std::mutex> get_lock() const {
14             std::unique_lock<std::mutex> loc(m_);
15             return loc;
16         }
17         void notify() const { cv_.notify_one(); }
18         void stop() { stop_ = true; }
19         void start() { stop_ = false; }
20         void join() { t_.join(); }
21         bool push_front(job& fun) { return sync_queue_.push_front(fun); };
22         bool push_back(job& fun) { return sync_queue_.push_back(fun); };
23         std::optional<job> pop_front() { return sync_queue_.pop_front(); };
24         std::optional<job> pop_back() { return sync_queue_.pop_back(); };
25     };

```

The choice to use the `relaxed` version of `synchro_queue` is motivated by its greater efficiency compared to other versions. The possible failure of an operation in the presence of concurrent access to the same cell is not a relevant issue in the context of the `threadpool`, since the design itself minimizes such situations: the `threadpool` inserts jobs at the back of the queue, each worker extracts its own jobs from the front and, only in case of inactivity, attempts to steal jobs from others queues from the back.

Moreover, the cost of checking whether an operation has succeeded would not be eliminated even by using other versions. In a finite-size structure like `synchro_queue`, it is always necessary to check that an insertion or removal operation does not fail due to a full or empty queue. Consequently, the `relaxed` version represents the best choice for the

purposes of this project.

It's important to not that there is no destructor in this class, and therefore no automatic resource management mechanism according to the *RAII* paradigm was implemented. The correct release of resources (threads, condition variables) is in fact delegated to the `threadpool` destructor, since, for design reasons, the `worker` class is not intended to exist independently outside of it.

Furthermore, the `push_*()` methods take their argument by reference since they act only as wrapper functions (intermediate steps); the `synchro_queue` then stores a copy of the input job, in accordance with common best practices in multithreaded programming.

2.3.2. `threadpool` class

The `threadpool` class provides a generic and extensible framework for parallel task execution based on a fixed set of worker threads. Its design separates task submission, scheduling, and execution, while supporting dynamic work redistribution through task stealing.

The class is templated on two policy types: a *scheduling strategy*, which determines how newly submitted jobs are initially assigned to workers, and a *stealing strategy*, which governs how idle workers attempt to retrieve jobs from other workers. This policy-based design allows for different scheduling and stealing heuristics to be employed without modifying the core architecture.

```
template <typename SchedulingStrategy = round_robin_scheduling,
          typename StealingStrategy = random_stealing>
class threadpool
```

API

```
threadpool(int size_queue, int n_worker)
```

Builds a thread pool with `n_worker` workers, each having a `synchro_queue` of size `size_queue`

```
threadpool(int size_queue)
```

Builds a thread pool with default number of workers given by `std::thread::hardware_concurrency()`, each having a `synchro_queue` of size `size_queue`

```
threadpool()
```

Builds a thread pool with default number of workers, each having a `synchro_queue` of size 256

```
int n_workers() const
```

returns the number of active workers in the thread pool.

```
int index_worker() const
```

Returns the index of the worker associated with the thread currently executing it.

Note: this allows a running task to query the worker executing it, enabling worker-aware task behavior.

```
int n_jobs()const
```

returns $\sum_{i=0}^{n_worker-1} count_job_ [i]$.

```
template <typename F, typename... Args>
auto send(F&& f, Args&&... args) -> std::future<decltype(f(args...))>
```

Wraps a function and its arguments into a job and submits it to the thread pool using the selected scheduling policy, returning a future for the asynchronous result.

```
template <typename F, typename Index>
requires std::is_same_v<std::invoke_result_t<F, Index>, void>
void parallel_for(Index start, Index end, F&& f)
```

Parallelizes the for-loop:

```
for(Index i = start; i!= end; ++i){f(i)}
```

wrapping each iteration into a separate job and submitting it to the threadpool.

Note: the `Index` type may be an integral type or an iterator.

```
template <typename F, typename Index, typename... Args>
void parallel_for(Index start, Index end, F&& f,
                 int granularity, Args... args)
```

Parallelizes the for-loop:

```
for(Index i = start; i!= end; ++i){f(i,index_worker,args...)}
```

by grouping `granularity` iterations into each job. Each job receives its own copy of the temporary objects passed through the variadic template. The body function `f` takes as input the loop index over the range, the index of the worker executing the job, and references to the temporary objects.

Note: the `Index` type may be an integral type or an iterator supporting the `-` and `+=` operators.

Further details on the handling of the possible values of `granularity` (including default selection when `granularity` is non-positive and clamping when it exceeds the iteration range), together with usage examples, are provided in Section 2.3.6.

```
template <typename F>
void parallel_for(int start, int end, F&& f, std::function<int(int)> incr)
```

Parallelizes the for-loop:

```
for(int i = start; i < end; i = incr(i)){f(i)}
```

wrapping each iteration into a separate job and submitting it to the threadpool.

```
template <typename F, typename... Args>
void parallel_for(int start, int end, F&& f, std::vector<int> granularities,
                Args... args)
```

Parallelizes the for-loop:

```
for(int i = start; i < end; ++i){f(i, index_worker, args...)}
```

by partitioning the iteration space into `granularities.size()` jobs, with job `j` covering `granularities[j]` iterations. Each job receives its own copy of the temporary objects passed through the variadic template. The body function `f` takes as input the loop index over the range, the index of the worker executing the job, and references to the temporary objects.

Note: the sum of all values in `granularities` must equal `end - start`.

```
template <typename Iterator, typename Value_type, typename F>
Value_type reduce(Iterator begin, Iterator end, Value_type init, F&& operation)
Takes a range [begin,end), an initial accumulator value init, and a binary
reduction operator operation. It returns the reduction of the range start-
ing from init by repeatedly combining elements with operation, exploiting
parallel execution across the thread pool.
```

threadpool private data members

This section describes the private data members of the `threadpool` class:

```
std::vector<std::shared_ptr<worker>> workers_;
std::deque<std::atomic<int>> count_job_;
int n_worker_;
int queue_size_;
mutable std::shared_mutex m_threadpool_;
std::condition_variable_any cv_threadpool_;
bool active_ = false;
std::unordered_map<std::thread::id, int> map_thread_worker_;
mutable std::shared_mutex m_map_;
StealingStrategy steal_policy_;
SchedulingStrategy schedule_policy_;
```

- `workers_`: this is the set of active workers in the thread pool. The choice to use `std::vector<std::shared_ptr<worker>>` is forced by the fact that the `worker` class, being non-movable due to the presence of `synchro_queue`, cannot be inserted directly into an `std::vector`.

Each worker is identified by its index within this vector.

- `count_job_`: this structure associates each worker, indexed in `workers_`, with a corresponding counter. The counters must be atomic as they are incremented whenever a job is inserted into the respective queue and decremented when a job is removed. The choice of using a `std::deque` is motivated by the desire to make accesses to `count_job_` as cache-friendly as possible, as well as by the inability to use `std::vector`, since atomic variables are non-movable, and `std::array`, because the number of workers is not known at compile time.
- `map_thread_worker_` and `m_map_`: a map that associates each workers thread identifier

with its corresponding worker index. This association allows the worker executing a given job to be identified at runtime. The associated mutex guarantees thread-safe insertion and lookup operations on the map, preventing data races during concurrent access.

- `m_threadpool_` and `cv_threadpool_`: synchronization primitives that ensure each worker starts actively searching for jobs in its own queue or in other workers queues only after all workers have been correctly initialized. Further implementation details are discussed in Subsection 2.3.3.
- `steal_policy_` and `schedule_policy_`: policy objects providing a `pick()` method used to select the worker index for stealing or job scheduling. Further implementation details are discussed in Subsection 2.3.4.

2.3.3. Implementation details

This section presents the implementations of the constructors, the destructor, and the `worker_loop` method executed by each worker thread. Since the thread pool implements task stealing, allowing each worker to access the queues of other workers, correct initialization and destruction are critical. In particular, premature access to the queues of non-initialized workers or delayed access to the queues of already destroyed workers would result in *undefined behavior* and may lead to *segmentation faults*.

The following code shows the implementation of the `threadpool` constructors.

```

1  threadpool(int size_queue, int n_worker):n_worker_(n_worker),queue_size_(size_queue){
2      std::unique_lock<std::shared_mutex> loc(m_threadpool_);
3      workers_.reserve(n_worker);
4      for (int i = 0; i < n_worker; i++) {
5          workers_.emplace_back(std::make_shared<worker>(size_queue,
6                                                         &threadpool::worker_loop, this, i));
7          count_job_.emplace_back(0);
8      }
9      active_ = true;
10     loc.unlock();
11     cv_threadpool_.notify_all();
12 };

```

The constructor begins by acquiring the thread pool's shared mutex in exclusive mode in order to safely initialize the internal state. The vector of workers is then reserved and populated, and each worker is constructed together with its local queue and execution thread. At the same time, the per-worker job counters are initialized to zero. Once all

workers have been correctly created, the `active_` flag is set to `true`. This flag is used in combination with the waiting mechanism implemented in `worker_loop()` to ensure that workers start actively searching for jobs and accessing other workers queues only after the entire initialization phase has completed. Finally, all worker threads waiting on the condition variable are notified.

The `void worker_loop(int i)` method defines the main execution routine of each worker thread. The parameter `i` identifies the index of the worker within the thread pool and uniquely determines the local resources (such as the job queue and counters) associated with the executing thread. Each worker thread first waits for the completion of the thread pool initialization, then registers its thread identifier, and finally enters the main execution loop, where it repeatedly executes available jobs until termination is requested.

The following pseudo-code describes the logical structure of the method.

Algorithm 2.1 worker loop

```

1: wait until all workers are initialized
2: register (thread id, worker index)
3: while the thread pool destructor has not been invoked do
4:   if local queue contains a job then
5:     execute job
6:   else if a victim worker can be selected then
7:     steal and execute job from victim
8:   else
9:     suspend execution and wait until a job arrives or shutdown is requested
10:  end if
11: end while

```

For completeness, the implementation of the method is shown below.

```

1  void worker_loop(int i) {
2      //Suspends execution until all workers are initialized.
3      //Shared mutex lock in read mode, so that all mutexes can perform
4      //steps concurrently
5      std::shared_lock<std::shared_mutex> lock_shared(m_threadpool_);
6      cv_threadpool_.wait(lock_shared, [this]() { return active_; });
7      lock_shared.unlock();
8      //Update map thread-safe
9      std::unique_lock<std::shared_mutex> loc_map(m_map_);
10     map_thread_worker_[std::this_thread::get_id()] = i;

```

```

11     loc_map.unlock();
12     int indx_steal = -1;
13     while (!workers_[i]->stop_) {
14         //Attempts to execute a job taken from the front of its own queue
15         if (try_do_(workers_[i]->pop_front(), i)) {
16             continue;
17         }
18         //Attempts to steal and execute a job from the back of another queue
19         indx_steal = steal_policy_.pick(count_job_, n_worker_);
20         if (indx_steal != -1) {
21             try_do_(workers_[indx_steal]->pop_back(), indx_steal);
22             continue;
23         }
24         std::unique_lock<std::mutex> loc(workers_[i]->m_);
25         workers_[i]->cv_.wait(loc, [&]() {
26             return count_job_[i].load(std::memory_order_acquire) > 0 ||
27                 n_jobs() > 0 || workers_[i]->stop_;
28         }); //Suspends execution if no jobs are in its own or other queues.
29         //later notified by send method
30         loc.unlock();
31     }
32 };

```

The helper method `try_do_` executes a job if the input `std::optional` is valid and updates the corresponding worker job counter:

```

bool try_do_(std::optional<job> j, int indx) {
    if (j) {
        (j.value())();
        count_job_[indx].fetch_sub(1, std::memory_order_release);
        return true;
    }
    return false;
}

```

Note: while jobs are available, whether in the local queue or stolen, the `worker_loop` method proceeds in a non-blocking manner, avoiding acquisition of its own mutex.

As anticipated, the destructor must ensure that all `worker_loop` instances have stopped, and therefore that no further concurrent access to the workers can occur, before proceeding with the destruction of the `threadpool`. By design, if the `threadpool` goes out of scope

before all jobs have completed, it waits for their completion before destroying itself.

The implementation of the destructor is shown below:

```

1  ~threadpool() {
2      // Non-blocking but unreliable waiting
3      while (n_jobs() > 0) { std::this_thread::yield(); }
4      // Blocking but reliable waiting
5      for (int i = 0; i < n_worker_; i++) {
6          while (!workers_[i]->sync_queue_.empty()) { }
7      }
8      for (int j = 0; j < n_worker_; j++) {
9          std::unique_lock<std::mutex> loc(workers_[j]->get_lock());
10         workers_[j]->stop();
11         workers_[j]->notify();
12     }
13     for (int j = 0; j < n_worker_; j++) { workers_[j]->join(); }
14 }

```

The destructor first performs a non-blocking but approximate waiting phase by repeatedly checking `n_jobs()` and yielding execution using `std::this_thread::yield()`, allowing threads that are still completing pending jobs to make progress without actively blocking the calling thread. Once the number of remaining jobs becomes approximately zero, the destructor switches to a blocking and reliable waiting phase, which relies on the `empty()` method of `synchro_queue` (described in Subsection 2.2.3), ensuring that no jobs remain unprocessed before proceeding with the shutdown. After all queues have been drained, the destructor signals termination to each worker by setting the corresponding stop flag and notifying the workers condition variable under mutex protection, allowing each worker thread to safely exit its `workerLoop()` execution. Finally, the destructor joins all worker threads, guaranteeing that all executions have completed before the destruction of the `threadpool` object.

Alternative design The user of the `threadpool` can explicitly control when a job is executed by invoking `get()` on the `std::future` associated with each submitted job. Consequently, the responsibility of ensuring that all jobs have completed before the `threadpool` object goes out of scope can be delegated to the user. Under this design choice, the initial waiting phase implemented in the destructor may be omitted and replaced by calls to the `clear()` method of each `synchro_queue`.

2.3.4. Scheduling and Stealing strategies

As anticipated, both scheduling and stealing strategies essentially reduce to selecting the index of the worker to which a job is submitted or from which a job is stolen. The `threadpool` configures these policies through the template type parameters, called `SchedulingStrategy` and `StealingStrategy`.

At present, two scheduling policies are provided:

- `round_robin`: jobs are dispatched in a cyclic manner; each call to `send()` targets the worker following the one selected by the previous submission, implementing a ring-buffer-like assignment.
- `least_load`: a job is submitted to the worker currently holding the smallest number of pending jobs in its queue.

Three stealing policies are available:

- `max_load`: selects the worker with the largest number of pending jobs.
- `random`: selects a victim worker uniformly at random among those with at least one pending job.
- `top_half_random`: selects a victim worker uniformly at random among the most loaded half of the workers that have at least one pending job.

From an implementation perspective, each policy is defined as a `struct` providing a `pick()` method that returns the selected worker index. In the following, we first discuss the implementation of the scheduling policies and then the stealing policies in detail.

Scheduling

The scheduling mechanism is based on the `send()` method, which takes as input a generic callable object and its arguments and returns a `std::future`. Following the design of `std::async`, the returned future allows the user to monitor the completion of the asynchronous execution and to retrieve the result by invoking `get()` at the desired point in the program.

We now present the implementation of `send()`.

shared pointer to a `packaged_task` that wraps a lambda capturing the function and its arguments by copy, and returning the function's output

```

1  template <typename F, typename... Args>
2  auto send(F&& f, Args&&... args) -> std::future<decltype(f(args...))> {
3      using return_type = decltype(f(args...));
4      std::shared_ptr<std::packaged_task<return_type()>> ptr_task =
5          std::make_shared<std::packaged_task<return_type()>>(
6              [fun = f, ... args_catturati = args]() mutable { return fun(args_catturati...); });
7      std::future<return_type> fut = ptr_task->get_future();
8      job j = [ptr_task]() { (*ptr_task)(); }; ← leads to a void() signature
9
10     int indx_worker = schedule_policy_.pick(count_job_, n_worker_);
11     std::unique_lock<std::mutex> lock(workers_[indx_worker]->get_lock());
12     bool flag = workers_[indx_worker]->push_back(j);
13     while (!flag) { flag = workers_[indx_worker]->push_back(j); }
14     count_job_[indx_worker].fetch_add(1, std::memory_order_release);
15     lock.unlock();
16     workers_[indx_worker]->cv_.notify_one();
17     return fut;
18 };

```

locks the mutex to ensure consistent notification of the insertion to the thread waiting on the cv in worker_loop. prevents a job from remaining unexecuted in the synchro_queue while the worker is sleeping

increment with release memory order to synchronize with subsequent acquire loads of the counter and ensure a view of the counter consistent with the state of the synchro_queue (who sees the counter increment also sees the job insertion)

First, the function is packaged so that its return value can be retrieved through an associated `std::future`. The scheduling policy then selects the index of the target worker, and the job is enqueued into the corresponding worker's local queue. Job submission is guaranteed by retrying the enqueue operation until it succeeds. Finally, the job counter of the selected worker is atomically incremented and the worker is notified via its condition variable, allowing it to resume execution if it was suspended.

Note: although the worker's mutex is locked at every send to the worker, this does not interfere with the execution of the jobs already present in its queue. As explained in

Subsection 2.3.3 dedicated to `worker_loop`, in fact, the worker does not pass through the mutex lock unless necessary.

The worker exhibits non-blocking behavior as long as popping jobs from its own queue or stealing from others succeeds.

Note: The `send` method does not wake up stealing workers; it only notifies the worker receiving the job. Stealing only occurs when a worker finishes its own jobs first. It does not make sense to trigger stealing during `send`. Enabling stealing on `send` would require notifying all workers for every job sent, which implies locking all worker mutexes for each `send`, leading to excessive synchronization.

We now present the implementation of the scheduling policies, together with the implementations of their respective `pick()` methods.

```
struct round_robin_scheduling {
    int indx_ = 0;
    // Returns the next worker index in round-robin order.
    int pick(std::deque<std::atomic<int>>& count_job, int n_worker) {
        int index = indx_;
        (indx_ == n_worker - 1) ? (indx_ = 0) : (indx_++);
        return index;
    }
};

struct least_loaded_scheduling {
    // Returns the index of the worker with the smallest job count.
    int pick(std::deque<std::atomic<int>>& count_job, int n_worker) const {
        int worker_indx = 0;
        int min_elem = count_job[0].load(std::memory_order_acquire);
        int current_el = 0;
        for (int j = 1; j < n_worker; j++) {
            current_el = count_job[j].load(std::memory_order_acquire);
            if (current_el < min_elem) {
                worker_indx = j;
                min_elem = current_el;
            }
        }
        return worker_indx;
    }
};
```

Stealing

The stealing policy is enforced through explicit calls to the `pick()` method of the `steal_policy_` object, which are performed inside the `worker_loop` execution. The structures implementing the available stealing policies are reported below.

```
struct max_load_stealing {
    // Returns the index of the worker with the largest job count.
    // If all counts are zero, returns -1 to indicate that no work can be stolen.
    int pick(std::deque<std::atomic<int>>& count_job, int n_worker) const {...};
};

struct random_stealing {
    mutable std::mt19937 gen_;
    random_stealing() : gen_(std::random_device {}()) {};
    // return random int in [0, size-1]
    int random_int(int size) const {...};
    // Returns a random index among the workers that have at least one job.
    // If no worker has jobs, returns -1.
    int pick(std::deque<std::atomic<int>>& count_job, int n_worker) const {...};
};

struct top_half_random_stealing {
    mutable std::mt19937 gen_;
    top_half_random_stealing() : gen_(std::random_device {}()) {};
    // return random int in [0, size-1]
    int random_int(int size) const {...};
    // Returns a random index among the workers in the top half of the busiest ones.
    // If no worker has jobs, returns -1.
    int pick(std::deque<std::atomic<int>>& count_job, int n_worker) const {...};
};
```

The `max_load` strategy is effective in the presence of significant workload imbalance, as it actively redistributes jobs among workers. However, it may lead to congestion, since multiple thieves tend to target the same heavily loaded worker, increasing contention on the corresponding `synchro_queue` mutex. The `random` strategy mitigates this congestion by uniformly distributing steal attempts across workers, thereby reducing contention and starvation. On the other hand, it does not explicitly address workload imbalance and may leave uneven job distributions unchanged. Finally, the `top_half_random` strategy represents a compromise between the two previous approaches. It preserves the rebalancing capability of `max_load` while limiting contention and reducing starvation through

randomized victim selection.

We conclude by presenting the implementation of the `pick()` method for the `top_half_random` stealing strategy.

```

1  int pick(std::deque<std::atomic<int>>& count_job, int n_worker) const {
2      // vector of pair: (index worker busy, number of job its queue)
3      std::vector<std::pair<int, int>> indxs;
4      int tmp_count_job = 0;
5      for (int k = 0; k < n_worker; k++) {
6          tmp_count_job = count_job[k].load(std::memory_order_acquire);
7          if (tmp_count_job > 0) { indxs.emplace_back(k, tmp_count_job); }
8      }
9      int size = indxs.size();
10     if (size == 0) { return -1; }
11     // sort from mostbusy to least
12     std::sort(indxs.begin(), indxs.end(), [](std::pair<int, int>& a,
13                                             std::pair<int, int>& b){
14         return a.second > b.second;
15     });
16     if (size < 4) { // avoid random generation if not need it
17         return indxs[0].first;
18     } else {
19         //randomly select index from first half of vector
20         return indxs[random_int(size / 2)].first;
21     }
22 };

```

2.3.5. Synchronization of count_job_

Let us now analyze in detail how `count_job_` synchronization takes place.

Consistent reading thanks to acquire-release Each store (increment in `send`, decrement in `try_do()`) is performed with `memory_order_release`, and each load with `memory_order_acquire`. This ensures that all operations on the `synchro_queue` of worker `i` corresponding to the operations on `count_job_[i]` are visible to subsequent reads of the `count_job_[i]` counter, thanks to the `synchronize-with` relationship established between the stores and loads.

Enforced Store-Load Ordering The only enforced ordering between the store and the load of `count_job_[i]` occurs between the increment performed in a `send` to worker `i` and the load executed during the check of the condition variable at the end of the while-loop in the `worker_loop`. This ensures that every job in the queue of a sleeping worker is eventually executed.

2.3.6. `parallel_for`

A for-loop is characterized by the range $r = [start, end)$ over which the index i varies and by the *body function* $f(i)$ executed for every value of $i \in [start, end)$.

The implemented `parallel_for` functions split the range r into n_{job} disjoint subsets r_k : r_k for $k \in 0, \dots, n_{job} - 1$ s.t. $r_i \cap r_j = \emptyset \forall i \neq j$, and send to the thread pool n_{job} jobs, each of which executes a subset of the for-loop corresponding to the subset of the range (the k -th job executes $f(i) \forall i \in r_k$).

The total number of jobs into which a for-loop can be divided is therefore variable: from a minimum of 1 (all iterations in a single job) to a maximum of $(end - start)$ (each single iteration of the loop corresponds to one job).

Trade-off There is a trade-off between balancing the workload among the workers and the overhead introduced by the thread pool. In case of reasons related to the operating systems thread management or imbalances in the workload among the different $f(i)$ a finer partitioning, and thus a larger number of jobs, allows through job stealing that the work time will be better balanced among the workers. However, a larger number of jobs is also associated with a higher number of sends, and thus a higher fixed cost due to the use of the thread pool.

Granularity The granularity represents the number of iterations that compose each job: granularity of the i -th job $= |r_i|$. A visual example of how granularity affects the dimensions of jobs can be seen in figure 2.6.

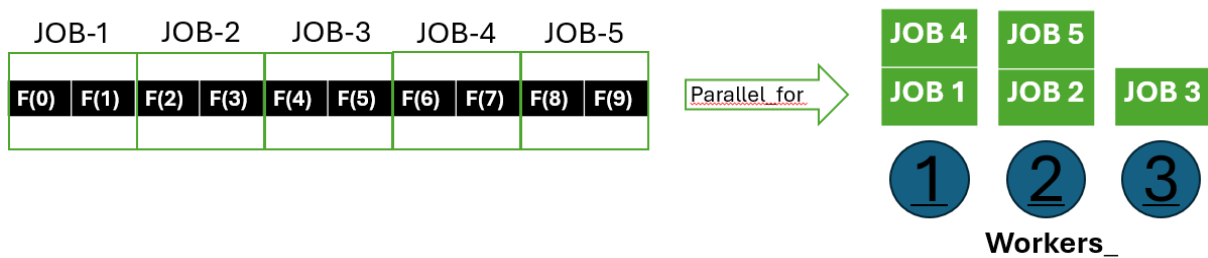


Figure 2.6: Visual example of how `parallel_for` works.

The division of the range into subsets can be achieved by providing, instead of granularity, also n_{job} .

Although the two approaches right appear identical, taking granularity as input was preferred. This is because the last job, composed of the remaining ($range \% n_{job}$) or ($range \% granularity$) iterations, will, in the latter case, be limited by granularity and thus smaller than the others. In contrast, when the number of jobs is fixed, the last job could contain more iterations than the others, potentially leading to load imbalance.

parallel_for methods

As shown in Section 2.3.2, several overloads of the `parallel_for` method are implemented. We now analyze in more detail the following overloads:

- `void parallel_for(Index start, Index end, F&& f)`, which uses a granularity value equal to 1. From this point onward, this overload is referred to as `gran1`.
- `void parallel_for(Index start, Index end, F&& f, int granularity, Args... args)`, which uses the granularity value provided as input. From this point onward, this overload is referred to as `gran_input`.

Differences between `gran1` and `gran_input` `gran1` is intended for the parallelization of for loops where the partitioning of the iteration range into jobs is performed manually, outside the thread pool.

For example, if we need to execute a loop:

```
Tmp_type tmp_obj; //not thread-safe
for(int i = 0; i < n; i++){BF(i); //use tmp_obj}
```

The loop should be interpreted as follows (note: $n = n_{job} * granularity$):

```
for(int i = 0; i < n_job; i++){
    Tmp_type tmp_obj;
    for(int j = 0; j < granularity; j++){
        BF(j+(i*granularity)); //use tmp_obj
    }
}
```

and therefore the parallel code becomes:

```
threadpool.parallel_for(0,n_job,[](int i){
    Tmp_type tmp_obj;
```

```

    for(int j = 0; j<granularity; j++){
        BF(j+(i*granularity)); //use tmp_obj
    }
});

```

This mechanism is instead handled directly by the overload of the `parallel_for` function that takes granularity as an input parameter `gran_input`.

```

Tmp_type tmp_obj;
threadpool.parallel_for(0,n,[](int i, int index_worker, Tmp_type& tmp_obj){
    BF(i); //use tmp_obj
}, granularity, tmp_obj);

```

The handling of temporary objects that are not thread-safe but modified concurrently is the same in both versions: each job creates its own copy, which is accessible to the iterations executed within that job.

`gran1` is sometimes more efficient because it allows for a more flexible job implementation, as each job corresponds exactly to the `F&& f` passed as input. On the other hand, `gran_input` is more straightforward to use, since it does not require the user to manually compute how the range is split.

An example illustrating the equivalent use of the two methods is shown in the parallelization of `grid_search` in Subsection 3.2.3.

Usage guidelines for `gran_input` If `f` is passed as input is a lambda function, the user should capture by reference only those variables that are safely accessed concurrently (thread-safe). Objects that would normally be captured by value should instead be passed explicitly as reference input parameters to the function `f`, and then supplied as arguments to `parallel_for`. This ensures that such objects are created once per job and avoids recreating them at every iteration.

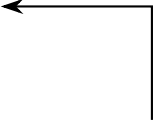
Example:

```

//sequential loop:
Thread_safe_type thread_safe_obj;
Tmp_type tmp_obj;
for(int i = start; i < end; i++){f(...);}

//parallelization:
std::vector<result_type> worker_partial_result(Tp.get_n_worker());
Thread_safe_type thread_safe_obj;
Tmp_type tmp_obj;
Tp.parallel_for(start,end,
    [&thread_safe_obj,&worker_partial_result]
    (int i, int indx_worker, Tmp_type& tmp_obj){
        f_forParallel(...)
    },
    granularity,tmp_obj);

```



`f_forParallel(...)` is the version of the sequential body function `f(...)` with the necessary modifications to work in parallel. (e.g., use of `indx_worker` to access the common vector in the position dedicated to the worker executing the job).

Input value of `granularity` This section illustrates how the input values of `granularity` are handled in the `gran_input` overload of `parallel_for`:

- **`granularity ≥ |range|` (= `end-start`):** all range in a single job (sequential execution).
- **`0 < granularity < |range|`:** creates `n_job` of `granularity` iterations (`n_job = |range| / granularity`).

If `last_iteration = |range| % granularity != 0`, two possible scenarios can occur:

- If `n_job` is multiple of `n_worker`, the last iterations are spread into the first job of the first `last_iteration` worker. (because all workers already have the same number of jobs).
- Otherwise, an extra job (with smaller granularity) is given to the first worker that has fewer jobs.

Examples:

- 3 workers (W0,W1,W2), range = [0,22), granularity = 5:

$n_job = 22/5 = 4$, not multiple of `n_worker`

→last job (2 iterations) assigned to W1.

- 3 workers (W0,W1,W2), range = [0,26), granularity = 4:

$n_job = 6$ multiple of `n_worker`

→spread remaining iterations: W0 and W1 get first jobs with granularity = 5.

- **granularity** ≤ 0 : default value used so that all workers have at most one job. This was implemented to ensure, if possible, the full utilization of the thread pool without considering the possibility of exploiting job stealing. In fact, there is no absolute optimal value for the granularity, as it is a trade-off between the fineness of the jobs and the minimization of send operations: an optimal value depends on the weight and number of the individual iterations.

Obtaining the return value Each body function `f` must be a `void` function that takes as input the index that scans the range defined as [start,end). This does not limit the possibility of obtaining return values: indeed, it is sufficient that the body function acts on references to external variables or objects. In this context, the use of the data member `map_thread_worker_`, accessed through the `index_worker()` method, is fundamental as it allows each worker, during the execution of its job, to identify its own index. This allows, for example, writing into dedicated positions of a shared vector used to store partial results (see, for instance, the parallelization of `grid_search` in Subsection 3.2.3). It should be emphasized, however, that in a multi-threading context, concurrent writes to shared data structures can lead to *data races* or inefficiencies related to cache usage (*false sharing*). These issues will be discussed in more detail in Subsection 3.2.2, where the `parallel_for` method will be used in real (non-simulated) contexts, and possible mitigation strategies will be proposed.

Why `thread_local` was avoided C++ allows defining `thread_local` variables, i.e. variables for which each thread has its own private copy accessible exclusively by that thread. Although this feature is used in several multi-threading frameworks for creating temporary objects, and it may seem that it avoids reconstructing the same object for each job as in our custom `parallel_for`, using `thread_local` variables is not appropriate in

this context. This is because the lifetime of `thread_local` variables coincides with that of the thread that owns them. In a `threadpool`, threads are created only once and remain active throughout the program's lifetime; consequently, `thread_local` variables would never be destroyed, leading to unnecessary memory consumption.

Implementation We now show the implementation of `parallel_for` in its simplest form:

```
parallel_for(int start, int end, F&& f)

1  template <typename F, typename Index>
2      requires std::is_same_v<std::invoke_result_t<F, Index>, void>
3  void parallel_for(Index start, Index end, F&& f) {
4      using return_type = void;
5      std::vector<std::future<return_type>> ret_fut; ←
6      if constexpr (std::is_integral_v<Index>) {
7          ret_fut.reserve(end - start);
8      }
9      for (Index j = start; j != end; ++j) {
10         ret_fut.emplace_back(this->send(f, j));
11     }
12     for (std::future<void>& fut : ret_fut) {
13         fut.get(); ←
14     }
15     return;
16 }
```

vector of futures to store the futures of jobs sent to the thread-pool

Call `get()` on all futures before the method ends to ensure the entire for-loop has completed

3 | Performance testing

In this chapter, we present the performance tests conducted to evaluate the behavior and efficiency of both the `synchro_queue` variants and the `threadpool`. We begin with tests performed in artificial scenarios, including micro-benchmarks designed to isolate specific performance characteristics. We then conclude with the evaluation of the thread pool applied to the parallelization of algorithms from the `fdaPDE` library, providing real-world performance results within a practical computational setting.

3.1. Ad hoc tests

To test the performance of the `synchro_queue` implementation, we implement a complete blocking wrapper of a `std::deque` that requires acquiring the mutex lock and holding it for the entire duration of the pop and push operations. In this way, any concurrent operation on the structure is safe because sequential access for data modification is guaranteed.

We expect the `synchro_queue` to perform better, as reducing the critical section mitigates the performance penalty due to thread *starvation*, as previously anticipated in Subsection 1.2.2. We show a snippet of the implementation of the blocking dequeue:

```
template <typename T> class concurrent_deque{
    using value_type= T;
private:
    std::deque<std::optional<value_type>> queue_;
    std::mutex m_;
public:
    ...
    void push_front(value_type t){
        std::lock_guard<std::mutex> loc(m_);
        queue_.push_back(t);
    }
    std::optional<value_type> pop_front(){
```

```

        std::lock_guard<std::mutex> loc(m_);
        if(queue_.empty())
            return std::nullopt;
        value_type ret=std::move(queue_.back().value());
        queue_.pop_back();
        return ret;
    }
    ...
};

```

3.1.1. synchro_queue tests

Single-threaded pop and push

We first verify the efficiency of the individual methods executed in single-thread. We measure the time required to perform N operations on the different types of queues containing `std::string`.

Test code:

```

using value = std::string;
/*type of queue*/ q;
auto start = std::chrono::high_resolution_clock::now();
for(int j=0; j<N; j++){q./*operation*/;}
auto end = std::chrono::high_resolution_clock::now();
auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end-start);

```

Problem data: N: 160000; value_type = `std::string`

Execution info: CPU: Intel(R) Core(TM) i7-8550U CPU @ 1.80GHz, 4 cores / 8 threads;

Compiler optimization: -O3;

Runs: 30

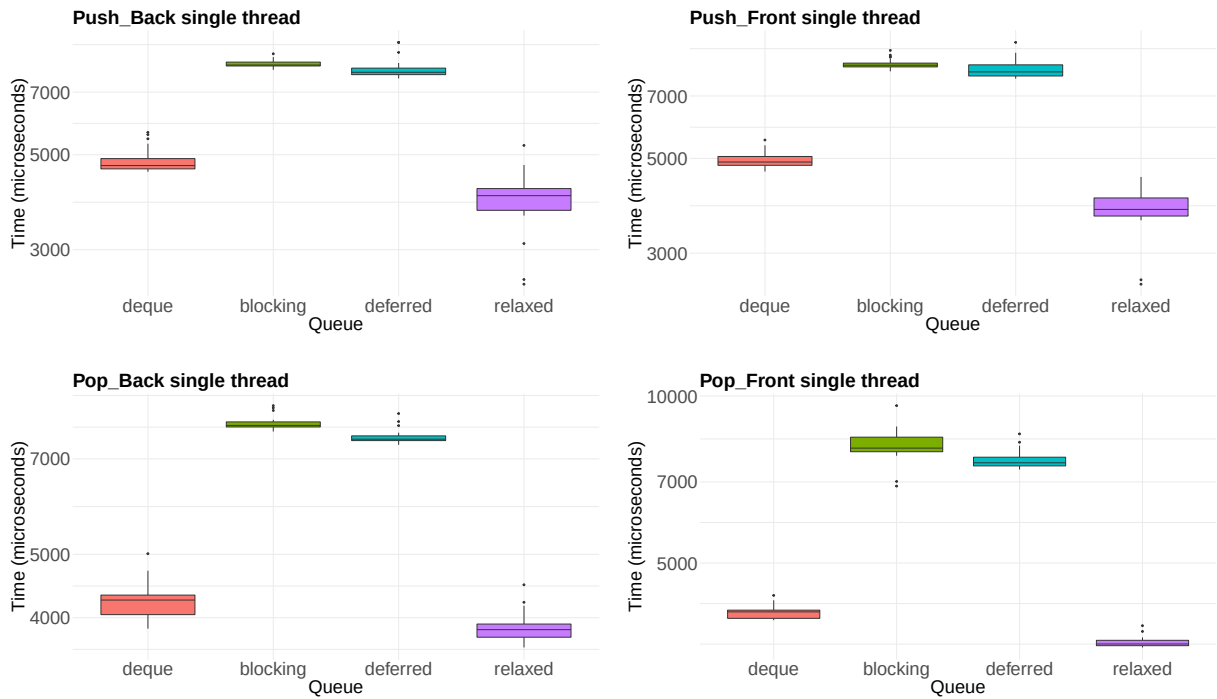


Figure 3.1: Box plots of the execution times of the single threaded push and pop tests, in $\log_{10}$ scale.

	deque	blocking	deferred	relaxed
push_back	4716	8098	7784	4012
push_front	4915	8244	7889	3794
pop_back	4246	7868	7500	3832
pop_front	4078	8040	7558	3564

Table 3.1: Medians of box plots for the single threaded push and pop operations [μs]

Result: the box plots of the execution times of this test are reported in Figure 3.1, while the medians of the execution times are reported in Table 3.1. The relaxed version shows overall better performance. The deferred and blocking specializations require locking two mutexes per operation, which introduces additional overhead.

Multi-threaded pop and push

We now test operations on queues executed with multiple threads. We show the measurements obtained by performing the same operation (pop/push front/back) N times on the

same shared queue, where each thread executes $N/n_threads$ operations.

Test code:

```
/*definition of function operations*/
/*queue type*/ q;
std::vector<std::thread> thread_pool;
auto start = std::chrono::high_resolution_clock::now();
for (int j=0; j<n_thread; j++){
    thread_pool.emplace_back(operations,std::ref(q),...);
}
for (int k= 0; k<n_thread; k++){
    thread_pool[k].join();
}
auto end = std::chrono::high_resolution_clock::now();
auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end - start);
```

To simulate multi-threaded execution, a vector of threads is used. Each thread is initialized via a function that performs N/n_thread operations of the same type on the shared queue. Note that the time measurement also includes the thread initialization and join phases. However, with a sufficiently large number of operations, the time associated with these phases can be considered negligible. Moreover, the goal of the test is not to evaluate the absolute execution time, but rather the relative comparison of the performance obtained with the different queue implementations.

Problem data: N : 160000; value_type = std::string; n_thread $\in [1,2,3,4,5,6,7,8]$

Execution info: CPU: Intel(R) Core(TM) i7-8550U CPU @ 1.80GHz, 4 cores / 8 threads;

Compiler optimization: -O3;

Runs: 30

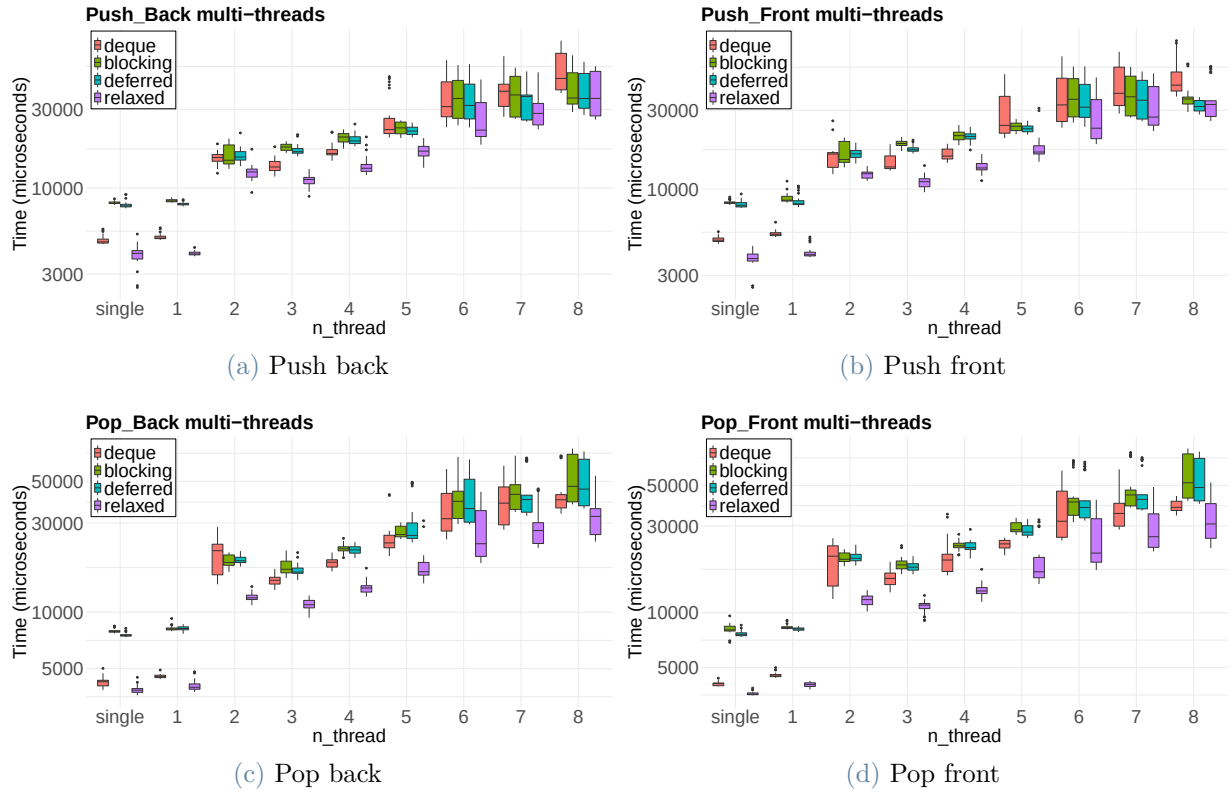


Figure 3.2: Box plots of the execution times of the multi-threaded push and pop tests, in $\log_{10}$ scale.

	push_back				push_front			
	deque	blocking	deferred	relaxed	deque	blocking	deferred	relaxed
single	4716	8098	7784	4012	4915	8244	7889	3794
1 threads	4977	8360	8015	4009	5357	8568	8140	4042
2 threads	15338	14756	15436	12544	16326	15118	16342	12398
3 threads	13433	17764	16628	11274	13803	18874	17297	11125
4 threads	16473	20533	19568	13460	16634	21596	20966	13704
5 threads	22732	24411	22792	17574	24316	24596	23822	17134
6 threads	31248	35000	31753	22486	32344	34972	31336	23368
7 threads	38754	36769	35965	28410	38014	36216	34515	27288
8 threads	46312	35245	34978	35043	42607	35316	33478	32538

Table 3.2: Median of box plots [μs] for multi-threaded push operations.

	pop_back				pop_front			
	deque	blocking	deferred	relaxed	deque	blocking	deferred	relaxed
single	4246	7868	7500	3832	4078	8040	7558	3564
1 threads	4541	8144	8209	3978	4509	8313	8186	4050
2 threads	21320	18439	18842	11974	20438	19646	19894	11798
3 threads	14884	16962	16398	10967	15432	18296	17788	11018
4 threads	18662	21864	21606	13484	19549	23500	23208	13182
5 threads	23952	26746	26410	16782	24648	28836	28420	16774
6 threads	31586	39113	35800	23225	31786	40812	37827	21258
7 threads	38258	42615	39990	27314	35078	44314	42490	26133
8 threads	40001	47036	45476	32512	40207	51671	48667	30700

Table 3.3: Median of box plots [μ s] for multi-threaded pop operations.

Results: the box plots of the execution times of this test are reported in Figure 3.1, while the medians of the execution times are reported in Tables 3.2 and 3.3. In a multi-threaded setting, all implementations suffer from starvation due to mutex contention. However, the `synchro_queue` mitigates this effect thanks to a shorter lock-holding time. Moreover, its performance further benefits from the fact that insertion and removal operations can effectively proceed in parallel, since all operations are identical and act on different elements of the queue. As a consequence, the `deferred` and `blocking` specializations, which performed worse than the `deque` in the single-threaded case, achieve performance comparable to the `deque` under multi-threading. The `relaxed` version, which was already the best performer in the single-threaded setting, remains the overall best also in the multi-threaded scenario.

Note: The goal of `synchro_queue`, in addition to ensuring correct thread-safe behavior, is to prevent performance degradation rather than to achieve speedup. Within the thread pool, job push and pop operations account for $o(\text{total_work})$ and are therefore not intended to be a source of parallel acceleration.

Random multi-threaded operations

In the previous tests, each operation was guaranteed to act on a different queue element, since all threads always performed operations of the same type. To test concurrent operations on the same queue elements as well, we run an experiment in which, starting from a half-full queue, each thread executes a sequence of N/n_{threads} random operations.

Each operation is selected according to a uniform distribution over the following options: `pop_front`, `pop_back`, `push_front`, and `push_back`.

Problem data: N : 160000; `value_type = std::string`; `n_thread` $\in [1, 2, 3, 4, 5, 6, 7, 8]$

Execution info: CPU: Intel(R) Core(TM) i7-8550U CPU @ 1.80GHz, 4 cores / 8 threads;

Compiler optimization: `-O3`;

Runs: 30

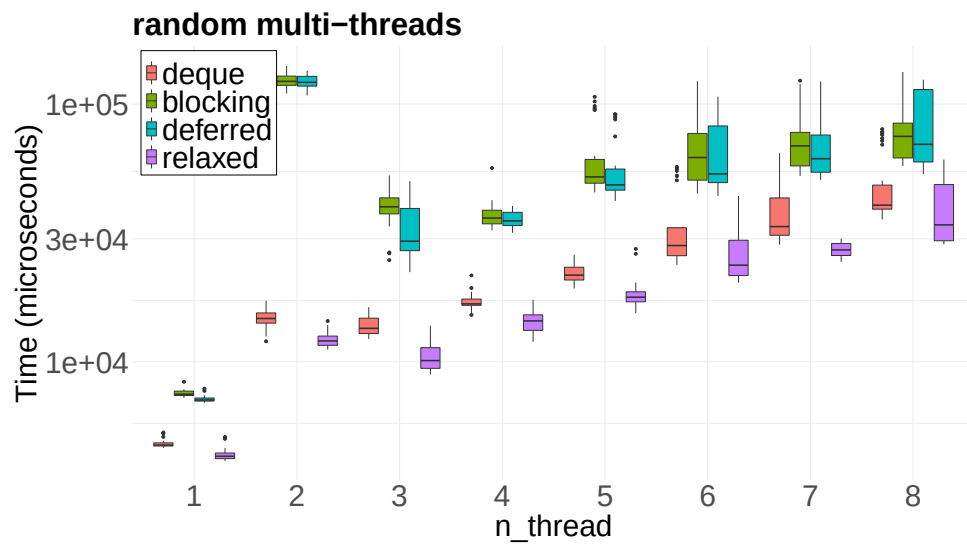


Figure 3.3: Box plot of the execution times for the random multi-threaded operations test

	deque	blocking	deferred	relaxed
1 threads	4754	7504	7120	4301
2 threads	14722	122481	121412	12046
3 threads	13486	40203	29362	10108
4 threads	16887	37508	36513	14580
5 threads	22300	52522	48764	18202
6 threads	28254	62010	53562	23705
7 threads	33469	68812	61361	28496
8 threads	40519	74917	69778	33986

Table 3.4: Medians of boxplots for random operation [μs]

Results: the box plots of the execution times of this test are reported in Figure 3.3, while the medians of the execution times are reported in Table 3.4. The **Relaxed** version remains the best-performing, while the **Deferred** and **Blocking** implementations show degraded performance. This is due to the overhead of contention when acquiring the internal mutexes of each element.

Comparison between Blocking and Deferred

Observe that in the tests the **Deferred** version performs slightly better than **Blocking**. This is due to the larger number of notifications in the **Blocking** methods, which are required to implement the additional `*_wait()` methods. Recall that each notification on a condition variable suspends thread execution, introducing overhead. It is therefore now clear why the two implementations were separated.

3.1.2. threadpool tests

In this subsection we test the performance of the methods implemented in the thread pool, in particular the scheduling and stealing logics: the performance of the thread pool is tested using different combinations of scheduling and stealing policies:

$(scheduling, stealing) \in \{round_robin, least_loaded\} \times \{max_load, random, top_half_random\}$.

We measured the scheduling and execution times of N unbalanced jobs by the thread pool, where half of the jobs were computationally lighter and the other half heavier, ensuring that half of the workers finish early and can perform work stealing. The computational weight of the jobs is described by the number of unit operations executed inside them (by unit operation we mean an operation like `a++`).

Test code:

```
fdapde::Threadpool<SchedulingStrategy,StealingStrategy> tp(4096,n_thread);
start = std::chrono::high_resolution_clock::now();
for( int i =0; i<n_job; i++){
    futs.push_back(std::move(tp.send(=)(int i)mutable{
        if(i%2 == 0){
            n_op_per_job *= 4;
        }
        int a= 0;
        for (int j = 0; j<n_op_per_job; j++){a++;}},i)));}
for(int i= 0; i<n_job; i++){futs[i].get();}
auto end = std::chrono::high_resolution_clock::now();
```

Problem data: n_job : 4096 ; $n_op_per_job = 1.5 \cdot 10^3 / 6 \cdot 10^3$;
 $n_thread \in [1,2,4,6,8]$; thread pool queue size: 4096

Execution info: CPU : Intel(R) Xeon(R) Silver 4208 CPU @ 2.10GHz,
 8 cores / 16 threads;

Compiler optimization: -O0;

Runs: 40

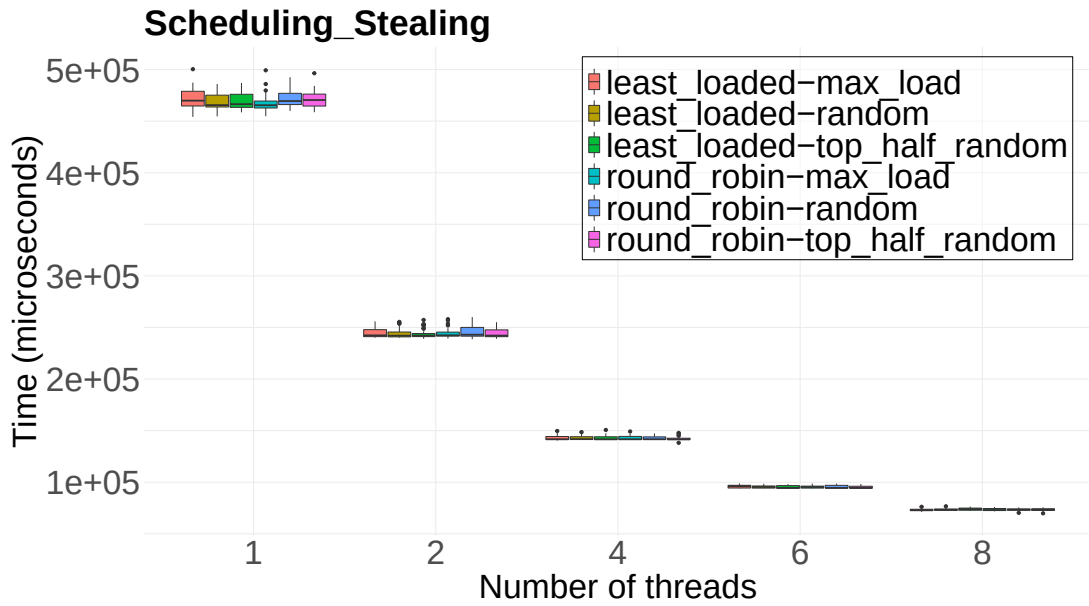


Figure 3.4: Box plots of the thread pool tests, in $\log_{10}$ scale.

	LL-ML	LL-R	LL-THR	RR-ML	RR-R	RR-THR
1 thread	469859.50	465511.50	466443.50	465419.50	469448.00	470444.00
2 threads	242337.50	242049.00	242144.00	242413.50	243008.50	242116.00
4 threads	141661.00	141756.50	141656.00	141648.50	141647.50	141532.50
6 threads	96171.50	95425.00	94536.00	95330.50	94476.50	94464.00
8 threads	72848.00	73316.50	73987.50	73097.50	73473.50	73181.00

Table 3.5: Box plot medians of the thread pool tests [μs]

Result: the box plots of the execution times of this test are reported in Figure 3.4, while the medians of the execution times are reported in Table 3.5. No significant differences are

observed across the tested combinations. The different scheduling and stealing strategies can therefore be considered equivalent in terms of the computational overhead they introduce. More extreme workload imbalance scenarios were not tested, as they are considered unlikely in the target use cases.

3.2. Scalability tests

3.2.1. Introduction to scalability testing

We first introduce the notation and theoretical background regarding the key performance indicators expected when executing parallel code. According to [9], *scalability* is defined as “the ability to handle more work as the size of the computer or application grows”.

The primary performance indicator of a parallel implementation is the *speedup*, defined as:

$$S = \frac{T_{\text{sequential}}}{T_{\text{parallel}}}$$

where $T_{\text{sequential}}$ and T_{parallel} represent the execution times of the sequential and parallel versions, respectively.

The theoretical upper bound on the achievable speedup is provided by Amdahl’s Law:

$$S_{\text{max}} = \frac{1}{s + \frac{p}{n}}$$

where s is the serial fraction of the program that cannot be parallelized, p is the parallelizable fraction (so that $s + p = 1$), and n is the number of processing units (which, in the context of multi-threading, n corresponds to the number of concurrent threads).

Two main types of scalability tests are considered:

- **Strong scaling:** The problem size is fixed while the number of threads increases. The goal is to observe how the execution time decreases as more parallel resources are employed.
- **Weak scaling:** Both the problem size and the number of threads increase, keeping the workload per thread constant. The aim is to verify whether the total execution time remains approximately constant as the system scales. The main performance indicator in this type of test is the *efficiency*, defined as:

$$E = \frac{T_{1,1}}{T_{N,N}}$$

where $T_{1,1}$ is the time to complete a work unit using one thread, and $T_{N,N}$ is the time to compute N of the same work units using N threads.

3.2.2. `parallel_for` tests

We test the performance of the `parallel_for` method by measuring the execution time of the element-wise addition of two matrices:

$A, B, C \in \mathbb{R}^N$ such that $C_{i,j} = A_{i,j} + B_{i,j}$ for all $(i, j) \in (0, N - 1) \times (0, N - 1)$.

The matrices are represented through `std::vector<std::vector<double>>`. The straightforward sequential implementation of the code consists of two nested for-loops that iterate over rows (outer) and columns (inner):

```
for(int i = 0; i<size; i++){
    for(int j=0; j<size; j++){
        C[i][j] = A[i][j] + B[i][j];
    }
}
```

We parallelize the column summation using the `parallel_for` method to replace the outer loop. We use the overload that takes granularity as input, allowing us to modulate the number of columns summed in each job. We start by using the default granularity value:

```
tp.parallel_for(0,size,[&](int i, int index_worker){
    for(int j=0; j<size; j++){
        C[i][j] = A[i][j] + B[i][j];
    }
},-1);
```

As a term of comparison, we use an equivalent parallel implementation based on OpenMP:

```
#pragma omp parallel for num_threads(n_thread) shared(A,B,C)
for(int i = 0; i<size; i++){
    for(int j=0; j<size; j++){
        C[i][j] = A[i][j] + B[i][j];
    }
}
```

Test parameters

Problem data: N : 8000; $n_thread \in [1,2,4,8,16,32,64,96]$; thread pool queue size:1024

Execution info: computer :cluster AP;

Compiler optimization: -O0; Runs: 20

In Figure 3.5 and 3.6 we report the box plots of the measurements (in microseconds) and the speedup plot, respectively. obtained using the median execution times as $T_{\text{execution}}$. Table 3.6 and 3.7 contain the values of the medians of the box plots and the speedup values, respectively.

Since the program is entirely parallelizable ($p=1$), the maximum theoretically achievable speedup with k threads is k .

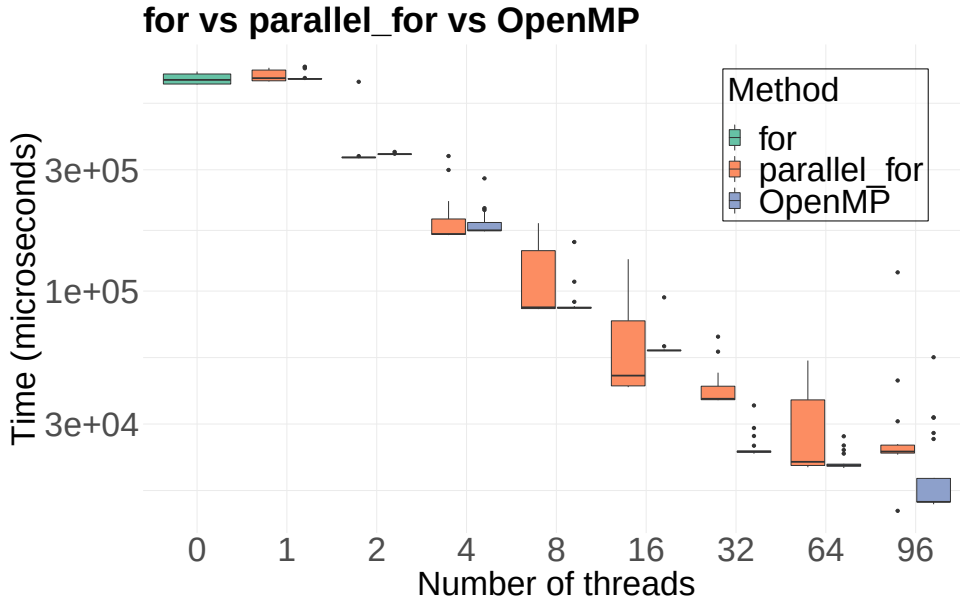


Figure 3.5: Box plots of the execution times of the parallel-for test, in $\log_{10}$ scale, with the default granularity value.

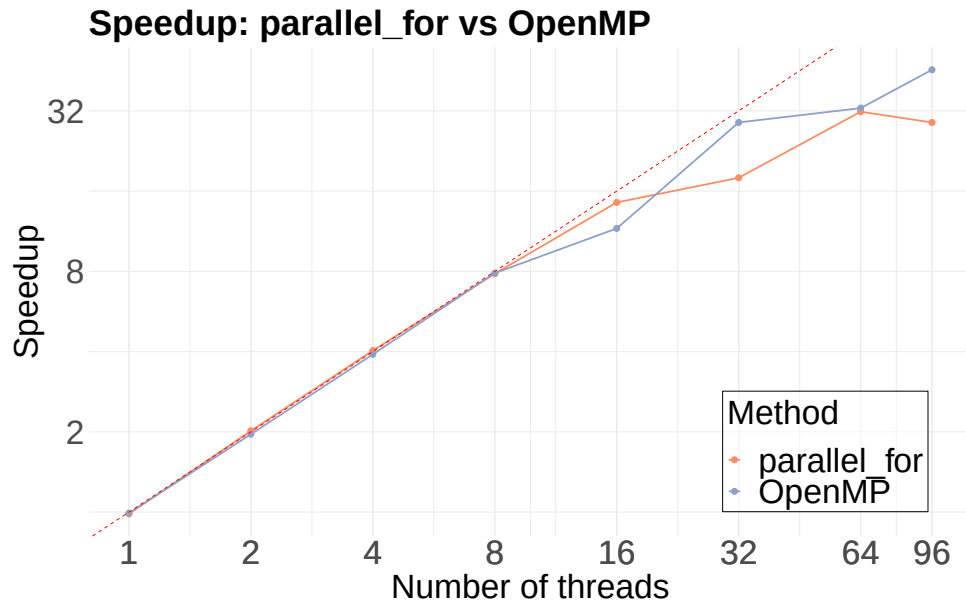


Figure 3.6: Speedup observed in the parallel-for test, in $\log_2 \log_2$ scale, with the default granularity value.

Thread	1	2	4	8	16	32	64	96
for	676990.0							
parallel_for	687725.5	335370.0	167685.0	86303.0	46654.0	37712.0	21307.0	23386.5
OpenMP	682849.5	345519.5	173326.0	85933.0	58401.0	23376.5	20659.5	14831.0

Table 3.6: Box plot medians [μs] of the parallel-for test, with the default granularity value.

Thread	1	2	4	8	16	32	64	96
parallel_for	0.98	2.02	4.04	7.84	14.51	17.95	31.77	28.95
OpenMP	0.99	1.96	3.91	7.88	11.59	28.96	32.77	45.60

Table 3.7: Speedup values of the parallel-for test, with the default granularity value.

Results: the thread pool exhibits performance comparable to OpenMP, suggesting that it makes efficient use of hardware resources when scheduling and executing tasks. However, a larger variance in execution times is observed for the thread pool compared to OpenMP.

Recall that the default granularity value assigns at most 1 job per worker, preventing any work stealing.

For this reason, we repeat the same test with a granularity value of 25, so that we have $\frac{8000}{25} = 320$ jobs. This larger number of jobs allows workers to perform work stealing, meaning that threads that finish their tasks earlier can help those that are still busy. Consequently, we expect a smaller variance in execution times.

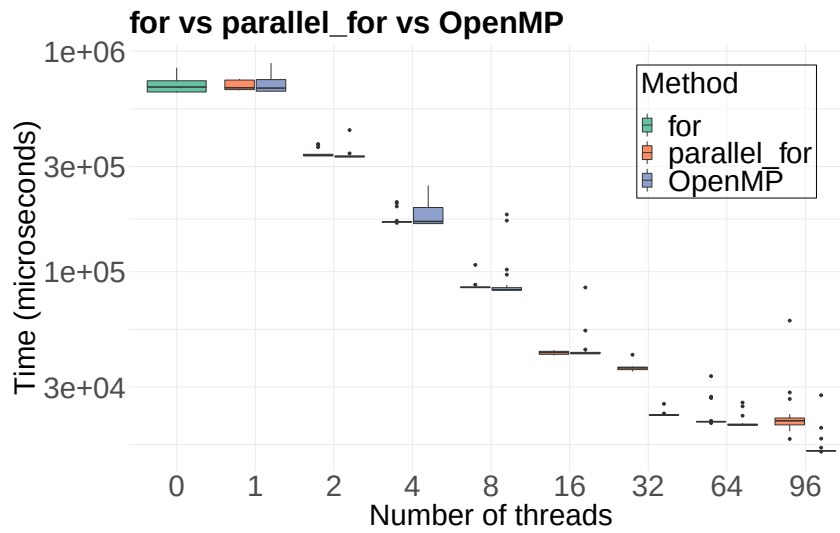


Figure 3.7: Box plots of the execution times of the parallel-for test, in $\log_{10}$ scale, with $\text{granularity} = 25$.

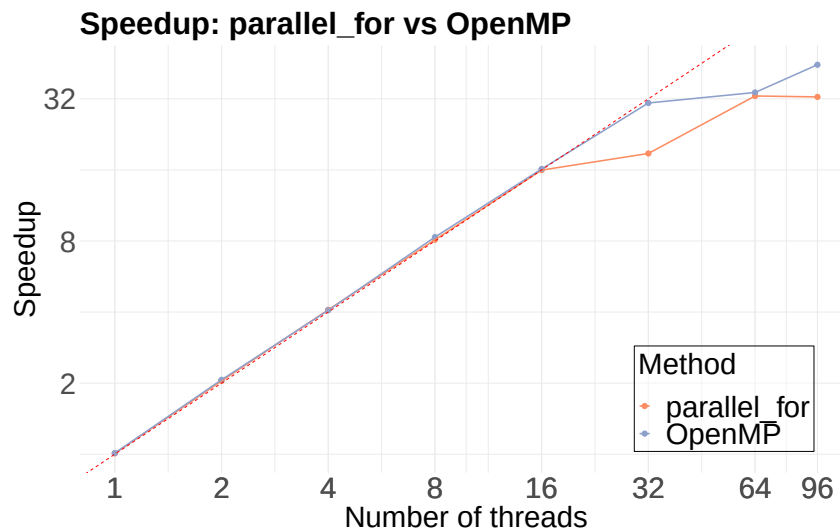


Figure 3.8: Speedup observed in the parallel-for test, in $\log_2 \log_2$ scale, with $\text{granularity} = 25$.

Thread	1	2	4	8	16	32	64	96
for	687726.5							
parallel_for	682058.0	336738.0	168239.0	84928.0	43060.0	36616.5	20900.0	21089.0
OpenMP	680064.0	332924.5	168950.5	82752.5	42598.5	22378.5	20199.0	15410.5

Table 3.8: Box plot medians [μ s] of the parallel-for test, with *granularity* = 25.

Thread	1	2	4	8	16	32	64	96
parallel_for	1.01	2.04	4.09	8.10	15.97	18.78	32.91	32.61
OpenMP	1.01	2.07	4.07	8.31	16.14	30.73	34.05	44.63

Table 3.9: Speedup values of the parallel-for test, with *granularity* = 25.

Results: the box plots of the execution times and the graph of the speedup are found, respectively, in Figure 3.7 and 3.8, while the numerical values of the box plot medians and speedup are found in Table 3.8 and 3.9 respectively. In this case, a significant reduction in variance is observed, as choosing a lower granularity value increases the number of jobs per worker, enabling the work-stealing mechanism to enhance overall performance. It is important to note that, in this test, the lambda functions passed to the `parallel_for` method only capture references to the matrices. This makes copying them both computationally lightweight and memory-efficient. Consequently, no overhead from an excessive number of send operations is observed.

3.2.3. Grid Search

In the `fdaPDE-core` library, the `GridSearch` class is defined in the following header file: `fdaPDE/src/optimization/grid_search.h`. It implements a grid search procedure that evaluates the objective function over a predefined grid in order to identify the minimum value and the corresponding *argmin*.

Here is a code snippet illustrating the class:

```
template <int N> class GridSearch {
private:
    ...
    vector_t optimum_;
```

```

    double value_;
    std::vector<double> values_;
    ...
public:
    ...
    vector_t x_curr;
    double obj_curr;
    ...
    template <typename ObjectiveT, typename GridT, typename... Callbacks>
    vector_t optimize(ObjectiveT&& objective, const GridT& grid,
                     Callbacks&&... callbacks) {
        ...
        // main loop of the algorithm
        for (std::size_t i = 1; i < grid.rows() && !stop; ++i) {
            grid_.row(i).assign_to(x_curr.transpose());
            obj_curr = objective(x_curr);
            stop |= internals::exec_eval_hooks(*this, objective, callbacks_);
            values_.push_back(obj_curr);
            if (obj_curr < value_) {
                value_ = obj_curr;
                optimum_ = x_curr;
            }
            stop |= internals::exec_stop_if(*this, objective);
        }
        ...
    }
    ...
};

```

The `optimize` method iterates through all points in the `grid` and evaluates the scalar field `objective` at each one, updating the minimum whenever a better value is found.

Parallel implementation of GridSearch

Since the evaluations of the grid points are independent from each other, the algorithm is well suited for parallelization. An overload of the `optimize` method has therefore been introduced, which specifies the parallel execution policy via an `execution::par` tag (following the design of `std::execution::par`)

```
namespace execution {
    struct execution_parallel {};
    inline constexpr execution_parallel par {};
}
```

and takes a threadpool and an optional granularity value as input.

```
template <typename ObjectiveT, typename GridT, typename Threadpool>
vector_t optimize(ObjectiveT&& objective, const GridT& grid,
                  execution::execution_parallel, Threadpool& Tp,
                  int granularity = -1)
```

Through the `parallel_for` method, each worker iterates in parallel over its assigned portion of the grid, finding the local minimum within its subset of points. At the end of the parallel processing, a sequential reduction phase is performed, during which the partial results from each worker are compared to determine the global minimum of the entire search.

We now observe the code that shows how the `parallel_for` method is used in the parallel implementation of the `optimize` method:

```
...
struct alignas(64) AlignedPair {
    double first = std::numeric_limits<double>::max();
    vector_t second;
};
// for storing the worker's partial result
int n_threads = Tp.n_workers();
std::vector<AlignedPair> value_optimum_workers(n_threads);
// manual division of the range into jobs
// (compute n_jobs and the granularity of the last job)
if(granularity == -1){
    granularity = (grid_.rows()/n_threads > 0) ? grid_.rows()/n_threads : 1;}
int granularity_last_job = grid_.rows()% granularity;
int n_job = (granularity_last_job == 0) ?
            grid_.rows()/granularity : grid_.rows()/granularity +1 ;
Tp.parallel_for(0,n_job, [&, this](int i){
    // create tmp and compute the job's granularity and sub-range
    int index_worker = Tp.index_worker();
    vector_t x_curr;
    double obj_curr =std::numeric_limits<double>::max();
```

```

vector_t x;
double obj = std::numeric_limits<double>::max();
int start = i*granularity;
int end = 0;
if(granularity_last_job != 0){
    end = (i != (n_job-1)) ? (i+1)*granularity : start+granularity_last_job;
}else{
    end = (i+1)*granularity;
}
// minimization of a portion of the grid assigned to the job
for(int j = start; j<end; j++){
    grid_.row(j).assign_to(x_curr.transpose());
    obj_curr = objective(x_curr);
    if (obj_curr < obj) {
        obj = obj_curr;
        x = x_curr;
    }
}
// optional minimal update of the worker
if(obj < value_optimum_workers[index_worker].first){
    value_optimum_workers[index_worker].first = obj;
    value_optimum_workers[index_worker].second = x;
}
});
/*reduce of value_optimum_workers[], min in value_ and argmin in optimum_*/
...

```

Note: In the parallel version, evaluations performed during the search are not tracked, unlike in the sequential algorithm where the `values_` vector is continuously updated. Moreover, early stopping of the search is not supported, since it would require communication mechanisms between workers that are not yet implemented in the thread pool.

Avoiding *false-sharing* Initially, an `AlignedPair` structure is defined, and a shared vector of `AlignedPair` is created. The vector is initialized with a number of elements equal to the number of workers, ensuring that each worker modifies only its own element and that concurrent updates are therefore thread-safe. The `AlignedPair` structure is used instead of `std::pair` to update the shared vector while avoiding *false-sharing*. In this

way, each thread accesses its own element in `value_optimum_workers[]` without loading adjacent elements into the same cache line. Thus, updates made by other workers are not perceived, preventing cache line invalidation and the resulting reload penalty.

Note: The cache line size is hard-coded as 64 bytes.

About the use of `gran1` Using `gran1` overload of `parallel_for` allows passing `F&&f` directly as the full implementation of the job. This makes it possible to first perform a local search by modifying local variables that live on the stack of the thread executing the job, and only after completing this search, possibly update the worker's minimum value in the shared vector.

The alternative implementation, using the overload of `parallel_for` that takes the granularity as input (`gran_input`), is the following:

```
...
// for storing the worker's partial result
struct alignas(64) AlignedPair {
    double first = std::numeric_limits<double>::max();
    vector_t second;
};
std::vector<AlignedPair> value_optimum_workers(Tp.n_workers());
long unsigned int start = 1;
Tp.parallel_for(start, grid_.rows(),
    [&Tp, &grid_, &value_optimum_workers, &objective]
    (long unsigned int i, int index_w, vector_t& x_curr, double & obj_curr){
// minimization of the grid portion assigned to the job
// directly in the worker's minimum
    grid_.row(i).assign_to(x_curr.transpose());
    obj_curr = objective(x_curr);
    if(obj_curr < value_optimum_workers[index_w].first){
        value_optimum_workers[index_w].first = obj_curr;
        value_optimum_workers[index_w].second = x_curr;
    }
}, granularity, x_curr, obj_curr);
/*reduce of value_optimum_workers[], min in value_ and argmin in optimum_*/
...
```

This implementation is indeed clearer and more straightforward, as it does not need to

handle all the details related to manually dividing the range both outside and inside the `parallel_for`. Here too, the temporary variables (`x_curr`, `obj_curr`) are created once per job; however, even though alignment avoids false sharing, performing writes to the shared vector is still more expensive than updating local variables on the stack, and this is why we prefer the first implementation, which is less immediate but more efficient.

Testing our GridSearch implementation

We perform strong and weak scaling tests, evaluating the parallel implementation against the sequential one by measuring the execution time of the sequential `optimize()` method and its parallel overload.

Note: To ensure a fair comparison between the sequential and parallel algorithms, in the sequential version the code lines updating the `values_` vector and checking for early stopping conditions have been commented out.

Test setting and code:

We create a grid of N points $\in \mathbb{R}^2$ uniformly distributed in the square of coordinates (x_i, y_i) such that $x_i, y_i \in [-5, 5]$.

As the objective function to minimize, we use the Rastrigin function:

$$f(x, y) = 20 + x^2 + y^2 - 10 (\cos(2\pi x) + \cos(2\pi y)).$$

```
fdapde::threadpool<fdapde::round_robin_scheduling,
                    fdapde::random_stealing> Tp(1024, n_thread);
fdapde::GridSearch<2> opt;
auto start = std::chrono::high_resolution_clock::now();
opt.optimize(rastrigin, grid, execution::par, Tp, granularity);
auto end = std::chrono::high_resolution_clock::now();
auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end-start);
```

Strong scaling test

The execution time on a grid of 12.8×10^6 points for the sequential version is compared with the parallel version using 1, 2, 4, 8, 16, 32, 64, and 96 threads.

Problem data: $N = 12.8 \cdot 10^6$; $n_thread \in \{1, 2, 4, 8, 16, 32, 64, 96\}$; thread pool queue size: 1024; granularity = -1

Execution info: Computer: [cluster AP](#)

Compiler optimization: -O3; Runs: 20

In Figure 3.9 we report the box plots of the measurements (in microseconds) and the speedup plot, obtained using the median execution times as $T_{\text{execution}}$. The box plot medians and the speedup values are found in Table 3.10.

Since the program is entirely parallelizable ($p=1$), the maximum theoretically achievable speedup with k threads is k .

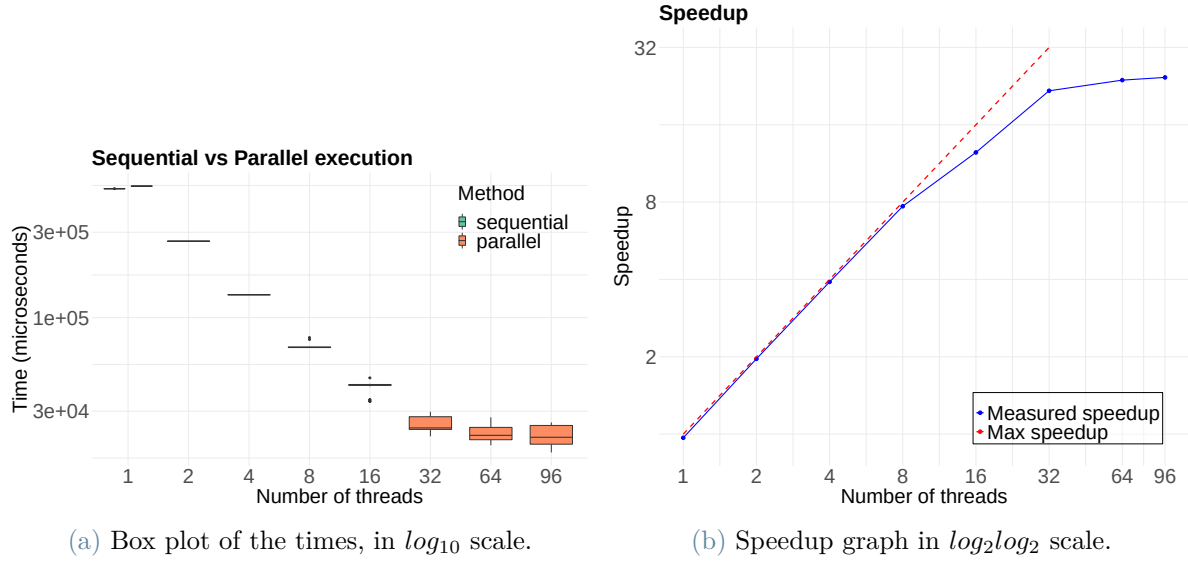


Figure 3.9: Plots of the results of the strong scaling test.

Thread	seq	1	2	4	8	16	32	64	96
Median [μs]	525125	542690	267442	134223	68136	42055	24208	22001	21472
Speedup	1.00	0.97	1.96	3.91	7.71	12.49	21.69	23.87	24.46

Table 3.10: Box plot medians [μs] and speedup of the strong scaling test.

Weak scaling test

The execution time of the parallel algorithm using 1 thread on a grid of 4×10^5 points is compared with runs using N threads on a grid of $N \times 4 \times 10^5$ points to maintain constant the work of each thread.

Problem data: $(N, \text{thread}) \in \{(k \cdot 4 \cdot 10^5, k) \text{ whith } k \in \{1, 2, 4, 8, 16, 32\}\}$, granularity = -1

Execution info: computer: [cluster AP](#)

compiler optimization: -O3; runs: 20

In Figure 3.10 we report the box plots of the measurements (in microseconds) and the efficiency plot, obtained using the median execution times. The box plot medians and speedup values are instead reported in Table 3.11.

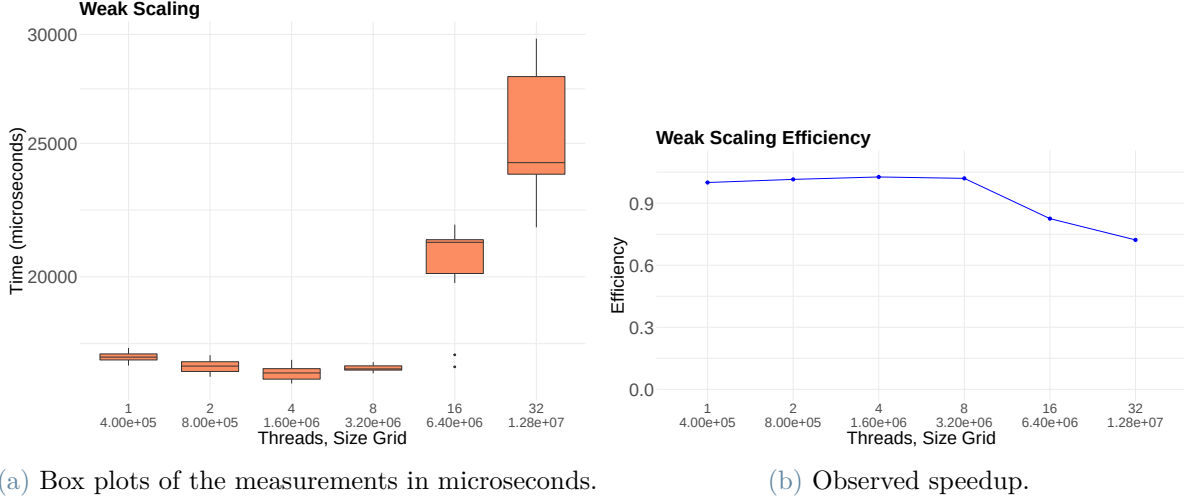


Figure 3.10: Plots of the results of the weak scaling test.

n_thread	size_grid	Median value	Efficiency
1	$4 \cdot 10^5$	17489.50	1
2	$8 \cdot 10^5$	17229.00	1.02
4	$16 \cdot 10^5$	17032.50	1.03
8	$32 \cdot 10^5$	17147.00	1.02
16	$64 \cdot 10^5$	21190.00	0.83
32	$128 \cdot 10^5$	24208.00	0.72

Table 3.11: Box plot medians [μ s] and speedup of the weak scaling test.

Results: the strong scaling test shows good results, with an excellent speedup up to 16 threads. Beyond this point, the speedup curve exhibits the expected flattening for 32, 64, and 96 threads, as the workload for each thread becomes too small to highlight further benefits of parallelization.

The weak scaling test also yields very good results, maintaining high efficiency across all the tested configurations.

Comparison between `gran1` and `gran_input` For completeness, we report the execution times measured for the parallel `optimize` implementation obtained using the

`gran_input` overload of `parallel_for`, corresponding to the alternative implementation illustrated in Section 3.2.3.

Problem data: $N = 12.8 \cdot 10^6$; `n_thread` $\in \{1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 14, 16\}$; thread pool queue size: 1024; granularity = -1

Execution info: CPU: Intel(R) Xeon(R) Silver 4208 CPU @ 2.10 GHz, 8 cores / 16 threads;

Compiler optimization: -O3; Runs: 30

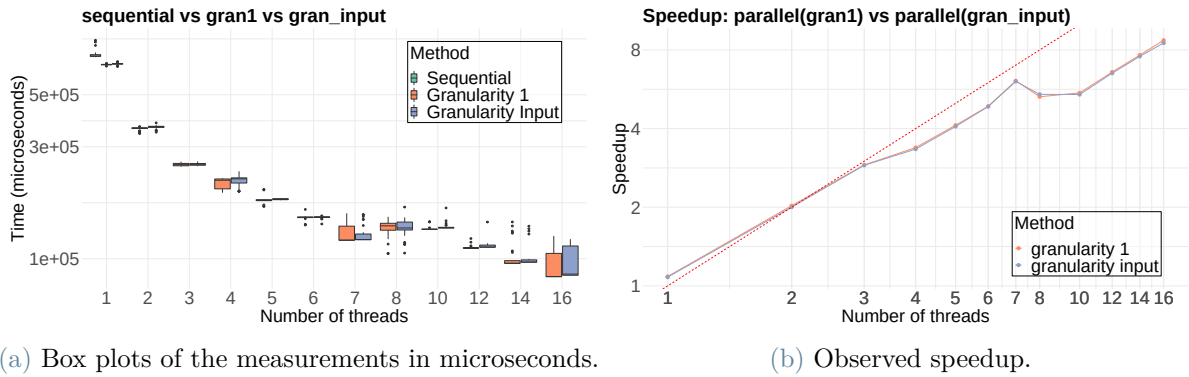


Figure 3.11: Plots of the test comparing `gran1` and `gran_input`.

Results: the test results are plotted in Figure 3.11. As anticipated, the version based on `gran_input` is slightly less efficient due to more frequent writes to a shared vector. However, the pairwise alignment of elements within the vector prevents *false-sharing*, and as a result the performance difference between the two approaches remains minimal.

3.2.4. Matrix assembly

In the `fdaPDE-core` library, the finite element method is implemented for the discretization and solution of PDEs. In particular, we focus on the `fe_bilinear_form_assembler` class, defined in the header file `fe_bilinear_form_assembler.h`, which implements the construction of the matrix derived from the bilinear form. For simplicity, we describe the specific case used to run the tests, i.e. the discretization of the Laplacian using linear finite elements on the unit square.

$$\text{find } u \in H_0^1(\Omega) \text{ s.t. } \begin{cases} -\Delta u = 0 & \text{in } \Omega, \\ u = 0 & \text{on } \partial\Omega \end{cases}$$

We obtain the weak formulation by multiplying by a test function $v \in H_0^1(\Omega)$ and integrating by parts:

$$\text{find } u \in H_0^1(\Omega) \text{ s.t. } \begin{cases} a(u, v) = 0 & \forall v \in H_0^1(\Omega), \\ a(u, v) = \int_{\Omega} \nabla u \cdot \nabla v \, dx \end{cases}$$

Let $\mathcal{T}_h$ be a triangulation of the domain Ω . We define the discrete space of linear finite elements as:

$$V_h = \{v_h \in C^0(\Omega) \mid v_h|_K \in \mathbb{P}_1(K), \forall K \in \mathcal{T}_h, v_h = 0 \text{ on } \partial\Omega\}.$$

The discrete weak formulation can be written as:

$$\text{find } u_h \in V_h \text{ such that } a(u_h, v_h) = 0 \quad \forall v_h \in V_h.$$

Let $\{\phi_i\}_{i=1}^N$ be a nodal basis of V_h , so that $\phi_i(p_j) = \delta_{ij} \forall (i, j) \in (0, N)$, where $p_1, \dots, p_N$ are the nodes of the triangulation $\mathcal{T}_h$. Writing u as a linear combination of the basis functions $u_h = \sum_{j=1}^N u_j \phi_j$, and using the basis functions as test functions, we obtain the following linear system:

$$\mathbf{A}\mathbf{u} = \mathbf{0},$$

where

$$\mathbf{A} = [A_{ij}], \quad A_{ij} = \int_{\Omega} \nabla \phi_j \cdot \nabla \phi_i \, dx, \quad \forall i, j = 1, \dots, N,$$

$$\mathbf{u} = [u_j]_{j=1}^N$$

We now present the `assemble()` method of the `fe_bilinear_form_assembler` class, which builds the global matrix $\mathbf{A}$.

```
Eigen::SparseMatrix<double> assemble() const {
    Eigen::SparseMatrix<double> assembled_mat(test_dof_handler()->n_dofs(),
                                              trial_dof_handler()->n_dofs());

    std::vector<Eigen::Triplet<double>> triplet_list;
    assemble(triplet_list);
    assembled_mat.setFromTriplets(triplet_list.begin(), triplet_list.end());
    assembled_mat.makeCompressed();
    return assembled_mat;
}
```

The assembly of the sparse matrix is performed starting from a vector of `Eigen::Triplet<double>`

computed with the overload method:

`void assemble(std::vector<Eigen::Triplet<double>>& triplet_list) const` which iterates over all mesh elements, through appropriate iterators defined in the library, and for each element K computes the local contributions

$$\int_K \nabla \phi_i \cdot \nabla \phi_j dx, \quad i, j \in \text{vertices}(K),$$

where ϕ_i and ϕ_j are the basis functions associated with the nodes of the element. Each local contribution is stored as an Eigen triplet, containing the indices i, j and the value of the corresponding integral, which represents the contribution of cell K to the matrix entry $A(i, j)$.

A schematic implementation of this method is shown below:

```
void assemble(std::vector<Eigen::Triplet<double>>& triplet_list) const {
    /* Initialize iterators begin and end of mesh cells */
    /* Prepare data structures for assembly */
    /* Create local fe_packet for element contributions */
    for (iterator it = begin; it != end; ++it) {
        /* Update fe_packet based on current cell */
        /* Integrate weak form over
           (i,j)-th basis function pairs of the cell*/
        /* Add local contributions to triplet_list */
    }
}
```

Note: The reduction phase of triplets with the same indices is then performed by the `setFromTriplets` method.

Parallel implementation of assemble

Since the computation of the partial contributions of each cell K is independent, the algorithm naturally lends itself to parallelization. An overload method has therefore been implemented, also specifying the parallel execution policy via the `execution::par`, which performs only the computation of the triplets in parallel and then assembles the matrix sequentially.

Parallel computation of the triplets: Using the `parallel_for` method, the mesh is divided among the workers, each of which computes in parallel the triplets related to the

cells assigned to it. It remains to explain how the triplets computed by each worker are inserted into the global vector, from which the matrix will then be built. A first solution envisioned each worker writing its own triplets into a dedicated vector, thus using a global vector of vectors of triplets. Although this strategy is *thread-safe*, it introduced additional overhead, since at the end of the computation phase it would be necessary to copy and merge the individual vectors into a single vector of triplets. To avoid this cost, a global vector of triplets of size `number_of_cells * number_of_triplets_per_cell` was initialized directly, containing empty elements. In this way, each worker can write directly into the global vector the triplets corresponding to the assigned cells, modifying the elements of the vector in the ranges `[cell_id * number_of_triplets_per_cell, (cell_id + 1) * number_of_triplets_per_cell]`. The writing is therefore *thread-safe*. Regarding efficiency, in particular avoiding *false sharing*, we rely on the homogeneous distribution of cells among the workers, which ensures sufficient spacing of accesses to the global vector (for example, with 100 cells and 5 workers, worker-0 starts from cell 0 and reaches 19, worker-1 starts from cell 20 and reaches 39, and so on).

The following two methods were therefore implemented:

```
template<typename Threadpool>
Eigen::SparseMatrix<double> assemble(execution::execution_parallel,
                                     Threadpool& Tp,
                                     int granularity = -1) const

template<typename Threadpool>
void assemble(std::vector<Eigen::Triplet<double>>& triplet_list,
              Threadpool &Tp, int granularity) const
```

The first method follows the logic of the corresponding sequential version, adds the computation of the total number of triplets in order to correctly initialize the `triplet_list` vector, and computes the triplets in parallel thanks to the second method. We now show the respective implementations:

```
template<typename Threadpool>
Eigen::SparseMatrix<double> assemble(execution::execution_parallel,
                                     Threadpool& Tp, int granularity = -1) const {
    Eigen::SparseMatrix<double> assembled_mat(test_dof_handler()->n_dofs(),
                                              trial_dof_handler()->n_dofs());
    int n_cell = this->Base::dof_handler_->triangulation()->n_cells();
    int triple_per_cell = n_trial_basis * n_test_basis;
```

```

    int tot_triple = n_cell * triple_per_cell;
    std::vector<Eigen::Triplet<double>> triplet_list(tot_triple);
    // computation of the triplet performed in parallel
    assemble(triplet_list, Tp, kk);
    // assembling the sparse matrix sequentially
    assembled_mat.setFromTriplets(triplet_list.begin(),
                                  triplet_list.end());
    assembled_mat.makeCompressed();
    return assembled_mat;
}

template<typename Threadpool>
void assemble(std::vector<Eigen::Triplet<double>>& triplet_list,
              Threadpool &Tp, int granularity) const{
    /* Initialize iterators begin and end of mesh cells */
    /* Prepare data structures for assembly */
    /* Create local fe_packet for element contributions */
    /* manual division of the range into jobs */
    Tp.parallel_for(0, n_job, [=, this, &Tp, &triplet_list](int ii) mutable{
        /* compute local begin iterator and cell-id:
        local_it, local_cell_id */
        for (int l = 0; l < granularity; l++) {
            /* Update fe_packet based on current cell */
            /* Integrate weak form over
            (i,j)-th basis function pairs of the cell */
            /* Thread-safe addition of
            local contributions to triplet_list */
            local_cell_id++;
            ++local_it;
        }
    });
    return;
}

```

Note that it is necessary to capture by copy in the lambda used in `parallel_for`, so that each worker has its own temporary objects and its own `fe_packet` to update during the computation of the integrals.

For the computation of the local begin iterators, the `operator+=` method was imple-

mented for `cell_iterator`, so that, starting from the `begin` iterator, the `local_begin` iterators can be computed with $o(1)$ computational cost:

```
iterator local_begin = begin;
local_begin += (ii*granularity);
```

Matrix assembly tests

We perform both strong and weak scaling tests on the triplet calculation part of the algorithm, measuring the execution time of the internal `assemble(...)` parallel overload function against the serial version. This phase corresponds to the parallelizable fraction of the algorithm, p , and therefore the maximum achievable speedup with k threads is k . Subsequently, we perform a strong scaling test on the overall algorithm by measuring the execution time of the external `assemble()` function for both the parallel and serial implementations. In this case, the expected speedup is limited by Amdahl's law, as described in subsection 3.2.1.

Test setting and code We consider the previously described example, namely the discretization of the Laplacian on the unit square.

```
using namespace fdapde;
Triangulation<2, 2> unit_square =
    Triangulation<2, 2>::UnitSquare(nodes_per_side);
FeSpace Vh(unit_square, P1<1>);
TrialFunction u(Vh);
TestFunction v(Vh);
//weak form of the laplacian
auto a = integral(unit_square)(dot(grad(u), grad(v)));
auto start = std::chrono::high_resolution_clock::now();
Eigen::SparseMatrix<double> A = a.assemble(execution::par, Tp, granularity);
auto end = std::chrono::high_resolution_clock::now();
auto duration =
    std::chrono::duration_cast<std::chrono::microseconds>(end - start);
```

For measuring the triplet calculation, the timer was placed within the internal `assemble(...)` function:

```
Eigen::SparseMatrix<double> assemble(execution::execution_parallel,
    fdapde::ThreadPool<fdapde::steal::random>& Tp,
    int granularity = -1) const
...
```



```

auto start = std::chrono::high_resolution_clock::now();
    assemble(triplet_list, Tp, granularity);
auto end = std::chrono::high_resolution_clock::now();
auto duration =
    std::chrono::duration_cast<std::chrono::microseconds>(end - start);
...
return assembled_mat;
}

```

The measurements of the full assembly and the triplet calculation were performed separately to exclude the timer overhead from the total `assemble()` execution time.

Note: to make the comparison fair, in the sequential version the vector of triplets was reserved with the total number of triplets, so as to avoid its reallocation during population.

Strong scaling test of triplet calculation

The execution time on a mesh of 1000×1000 nodes for the sequential version is compared with the parallel version using 1,2,4,8,16,32,64, and 96 threads.

Problem data: Mesh nodes: 1000×1000 , granularity: -1

Execution info: CPU: [cluster AP](#)

Compiler optimization: -O3; Runs: 20

In Figure 3.12 we report the box plots of the measurements (in microseconds) together with the corresponding speedup plot, while the medians of the box plots and the values of the speedup are reported in Table 3.12. The purely sequential implementation is slower than the serial execution obtained by using the thread pool with a single thread. Since our goal is to evaluate the performance of the thread pool rather than the algorithm itself, the reported speedup is computed by taking the onethread thread pool execution as the reference time, i.e., $T_{\text{serial}} = T_{\text{thread_1}}$.

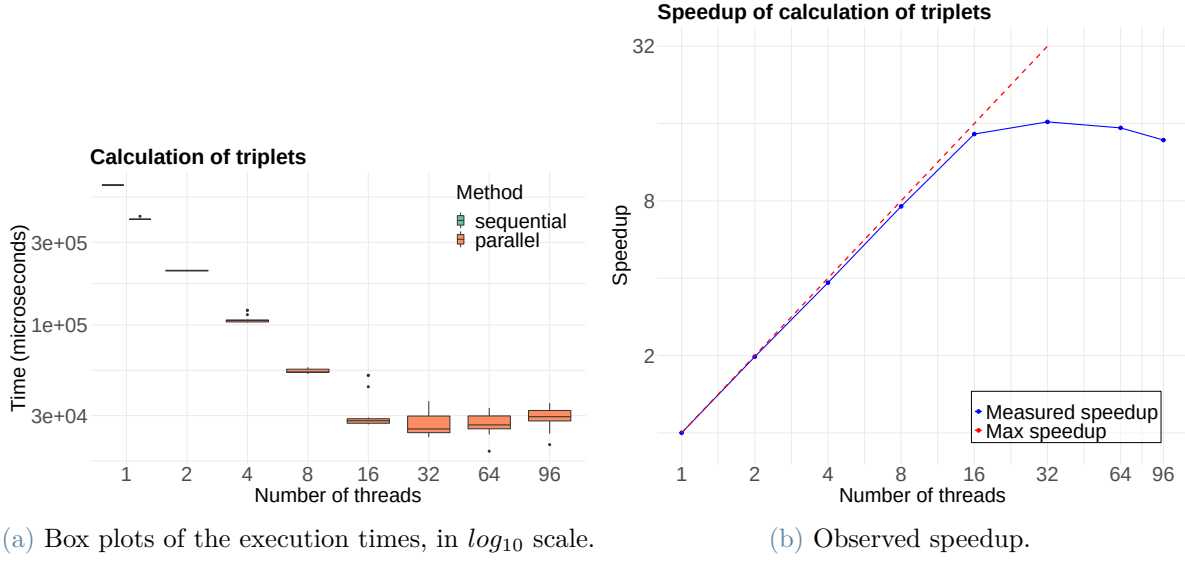


Figure 3.12: Plots of the strong scaling test results.

Thread	seq	1	2	4	8	16	32	64	96
Median [μs]	643935	407945	206188	106224	53596	28028	25132	26522	29580
Speedup	1.00	1.00	1.98	3.84	7.61	14.55	16.23	15.38	13.79

Table 3.12: Box plot medians [μs] and speedup for the strong scaling test.

Weak scaling test of triplet calculations

The execution time of the parallel algorithm using 1 thread on a mesh of 250×250 nodes is compared with that obtain from using 4 and 16 thread on mesh of 500×500 and 1000×1000 to keep the same workload on each thread.

Problem data: $(\text{mesh_nodes}, \text{thread}) \in \{((\sqrt{k} \cdot 250)^2, k) \text{ with } k \in \{1, 4, 16\}\};$
granularity = -1

Execution info: computer: [cluster AP](#)

compiler optimization: -O3; runs: 20

In Figure 3.13 we report the box plots of the measurements (in microseconds) together with the corresponding speedup plot, while the medians of the box plots and the values of the speedup are reported in Table 3.12.

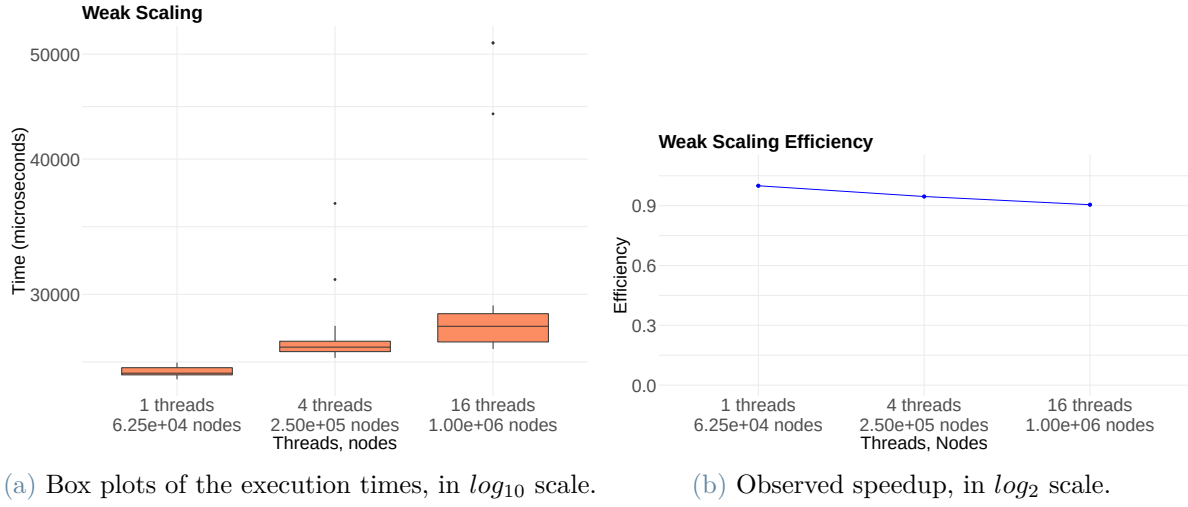


Figure 3.13: Plots of the weak scaling test results.

n_thread	mesh nodes	Median [μ s]	Efficiency
1	250^2	25366.00	1
4	500^2	26812.00	0.95
16	1000^2	28028.50	0.91

Table 3.13: Box plot medians [μ s] and speedup for the weak scaling test.**Strong scaling test of `assemble()`**

The execution time of the external `assemble()` method (i.e. the complete one) on a mesh of 1000×1000 nodes for the sequential version is compared with the parallel version using 1,2,4,8,16,32,64, and 96 threads.

Problem data: Mesh nodes: 1000×1000 , granularity: -1

Execution info: CPU: [cluster AP](#)

Compiler optimization: -O3; runs: 20

In Figure 3.14 we report the box plots of the measurements (in microseconds) together with the corresponding speedup plot, while the numerical results of the test are reported in Table 3.14. As in the triplet calculation test, the reported speedup is computed by taking the singlethread thread pool execution as the sequential time.

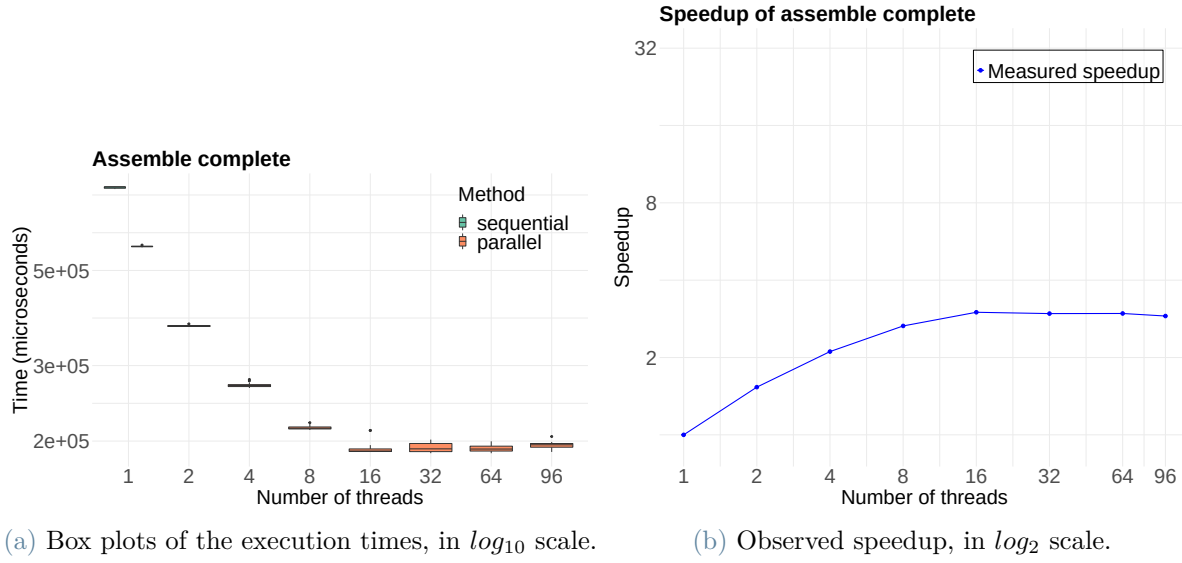


Figure 3.14: Results of the strong scaling test of `assemble()`

The execution of the triplet calculation on thread 1, denoted by p , takes $4.079 \cdot 10^5 \mu s$, while the execution of the full assembly on thread 1, denoted by $p + s$, takes $5.687 \cdot 10^5 \mu s$. From these measurements we obtain $p = 0.717$, and using Amdahl's law we can compute the maximum achievable speedup.

Thread	seq	1	2	4	8	16	32	64	96
Median [μs]	783356	568699	370836	269654	214289	189584	191831	191634	196061
Speedup		1	1.53	2.11	2.65	3.00	2.96	2.97	2.90
Max Speedup		1	1.56	2.16	2.69	3.05	3.28	3.40	3.45

Table 3.14: Box plot medians [μs] and speedup over thread 1 for the strong scaling test of `assemble()`.

Results: the strong scaling test of the triplet calculation shows very good results, with excellent speedup. As expected, the speedup curve flattens physiologically for thread counts above 16, since the workload per thread becomes too small for the benefits of parallelization to be clearly observable. Consequently, the complete assembly also exhibits excellent strong scaling behavior, with a speedup close to the theoretical maximum predicted by Amdahl's law.

In addition, the weak scaling test of the triplet calculation yields very good results, maintaining high efficiency.

3.2.5. Effect of granularity

This section analyzes the impact of using a granularity value different from the default one adopted so far in the `GridSearch` and `assemble` benchmarks. Recall that the default granularity assigns at most one job to each worker. While this configuration fully exploits the thread pool, it does not allow job stealing. Finer granularity values, on the other hand, enable stealing at the cost of an increased number of send operations and, consequently, more frequent copies of the callable `F&& f` passed to the `parallel_for` method.

We compare the performance obtained with the default granularity against a configuration in which each worker is assigned 10 jobs, for both benchmarks considered: `GridSearch` and `assemble` (triplet calculation only).

Effect of granularity on GridSearch

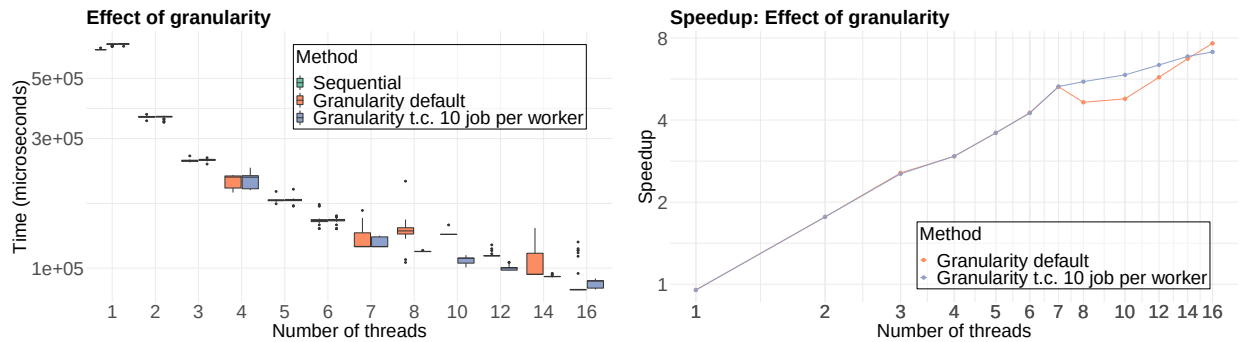
Problem data: Grid size: $12.8 \cdot 10^6$

Execution info: CPU: Intel(R) Xeon(R) Silver 4208 CPU @ 2.10GHz, 8 cores / 16 threads;

Compiler optimization: -O3; runs: 30

In Figure 3.15 we report the execution-time and speedup plots comparing the parallel overload of `optimize` obtained using first the default granularity, and then a granularity such that each worker receives 10 jobs. The numerical results are instead reported in Table 3.15.

```
opt.optimize(rastrigin, grid, execution::par, Tp,
             -1); // default granularity
opt.optimize(rastrigin, grid, execution::par, Tp,
             grid_sizes/(n_thread*10)); // 10 jobs per worker
```



(a) Box plots of the execution times, in $\log_{10}$ scale.

(b) Observed speedup, in $\log_2$ scale.

Figure 3.15: Plots of the result of the test on `GridSearch` with varying granularity.

Thread	1	2	3	4	5	6	7	8	10	12	14	16
Median sequential	700889	–	–	–	–	–	–	–	–	–	–	–
Median g.default	668386	360728	248992	216385	177506	149736	120225	137140	133130	111028	94907	83337
Median g.10jpw	669366	360968	250708	216226	177547	150127	120084	115164	108785	100058	93091	89590
Speedup g.default	0.95	1.77	2.56	2.94	3.59	4.26	5.30	4.65	4.79	5.74	6.71	7.65
Speedup g.10jpw	0.95	1.77	2.54	2.95	3.59	4.24	5.31	5.53	5.86	6.37	6.84	7.11

Table 3.15: Box plot medians [μ s] and speedup for the test on GridSearch with varying granularity.

Results: The overhead introduced by the additional send operations is negligible. The benefits of job stealing become evident for more than 8 threads, due to the activation of hyper-threading, a hardware feature that allows multiple logical threads to run on the same physical core. In this regime, increased contention for shared core resources makes it more likely for some workers to lag behind, thereby creating opportunities for effective work stealing.

Effect of granularity on triplet calculation

Problem data: Mesh nodes: 1000×1000

Execution info: computer: Intel(R) Xeon(R) Silver 4208 CPU @ 2.10GHz, 8 cores / 16 threads;

Compiler optimization: -O3; Runs:30

In Figure 3.16 we report the execution-time and speedup plots comparing the parallel overload of `assemble`, measured only on the triplecomputation phase (which is fully parallelizable), while numerical results are reported in Table 3.16. The results refer to executions performed first with the default granularity and then with a granularity such that each worker receives 10 jobs. (The speedup is computed with respect to the execution time of the parallel implementation run on a single thread, rather than the sequential version, as done previously.)

```
Eigen::SparseMatrix<double> A = a.assemble(execution::par,Tp,-1);//default gran.
```

```
int tot_cells = ((nodes_per_side-1)*(nodes_per_side-1)*2);
```

```
Eigen::SparseMatrix<double> A = a.assemble(execution::par,Tp,  
tot_cells/(n_thread*10));//10 jobs per worker
```

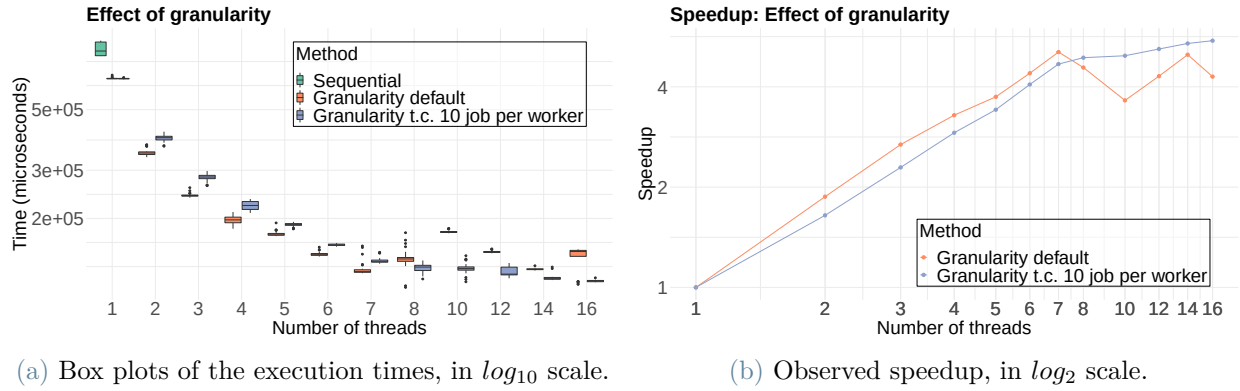


Figure 3.16: Plots of the results of the test on the effect of granularity on triplet assembly.

Thread	1	2	3	4	5	6	7	8	10	12	14	16
Median sequential	649615											
Median g.default	649615	347018	242080	197592	174046	147877	127741	142180	178428	150902	130092	151359
Median g.10jpw	649254	394786	283537	223055	190252	159836	138799	132796	131042	125084	120375	118080
Speedup g.default	1.00	1.87	2.68	3.29	3.73	4.39	5.09	4.57	3.64	4.30	4.99	4.29
Speedup g.10jpw	1.00	1.65	2.29	2.9	3.4	4.06	4.68	4.89	4.96	5.19	5.40	5.50

Table 3.16: Box plot medians [μ s] and speedup for the test of the effect of granularity on triplet assembly.

Result: The overhead introduced by the presence of a larger number of jobs becomes more evident in this case. As in the grid-search experiment, however, the benefit of work-stealing becomes noticeable once hyperthreading is activated.

Note: In both scenarios, work-stealing becomes particularly significant when the number of worker threads in the thread pool exceeds the number of available CPU cores. In this situation, the likelihood that some threads fall behind, due to OS-level scheduling and execution delays, increases, and work-stealing effectively mitigates the resulting imbalance. Conversely, no benefit is observed when the number of threads is lower than the number of cores, since in both tests the iterations of the parallelized for loops are perfectly balanced (each iteration has the same computational cost). If, however, the loop inherently contained imbalanced iterations, the advantage of work-stealing, and thus of using a finer granularity, would be apparent even when the number of worker threads is lower than the number of cores.

4 | Execution instructions

4.1. Requirements

The latest version of the code can be downloaded from:

<https://github.com/simonesolidoro/fdaPDE-core/tree/develop>

Compilation of the code requires the following:

1. `g++-14`
2. Eigen, an existing requirement for the fdaPDE library, used in our parallel implementation of matrix assembly and grid search.
3. OpenMP, parallel computing framework used for benchmarking.

The path of the required dependencies must be included in the `$PATH` system variable, or specified in the Makefiles.

4.2. Compilation

The test programs can be found in `test/src/test_multithreading`. This folder contains all the tests that were created during the development of this code to test the new functionalities we introduced. This folder contains two subfolders:

- `Test_functional` contains tests that were used during the development of the library, to check that the features we introduced were functioning as expected. These are the tests presented in Section 3.1.
- `Test_performance` instead contains performance benchmarks for several features of our code, which were introduced in Section 3.2.

To each of the tests is associated a Makefile, used to compile all the `.cpp` files inside the folder. If the `$PATH` variable does not contain the paths to all the dependencies, the Makefile must be updated to include them adding `-I/path/to/dependency` inside

CXXFLAGS. Running `make all` in the command line compiles all the files inside the folder, while `make clean` removes all the executables.

4.3. Execution

The tests can be executed in multiple ways: the compiled executable can be ran directly, making sure to include any required command line argument. This way, the time the test took to complete is displayed in the command line.

Another possibility, available for the tests found in **Test_performance**, is to run one of the included shell scripts: they can be used to perform directly multiple tests if it is required for benchmarking, by specifying the number of runs when executing the shell script. In this case, the output of the program is saved to a file whose name is specified inside the script.

Finally, in order plot the results of the performance benchmarks saved in text files, R scripts are provided.

5 | Future developments

The structure implemented so far, while functional, represents only a starting point and can be improved in several aspects. In this chapter, we present some ideas that could guide future work aimed at extending and enhancing the thread pool.

5.1. Lock-Free `synchro_queue`

As discussed in Section 2.2, the current `synchro_queue` is a blocking structure which, despite minimizing the critical section, limits concurrency by serializing the beginning of each operation. Implementing a lock-free queue, introduced in Section 1.2.2, which uses only atomic variables and compare-and-exchange operations, would enable more efficient job submission and retrieval. Lock-free FIFO queues, such as the one presented in [7], are already non-trivial to implement. Extending these structures to the more general multiple-producer multiple-consumer (MPMC) case is even more challenging, as it requires ensuring safe concurrent access on both ends of the queue for different reasons.

5.2. Worker Synchronization: Barriers and Fork-Join

Currently, the thread pool consists of independent workers that do not coordinate with each other. There is no mechanism allowing a worker to delegate part of its workload to other workers within the pool and then wait for its completion, which is a key feature of the fork-join model that underlies many parallel-for implementations. Enabling this functionality would require modifications to the design of the thread pool, including the introduction of barriers and counters to allow workers to wait for, and communicate, the completion of jobs by other workers.

5.3. FEM Matrix Assembly

In addition to improvements related to the `threadpool` structure itself, there are several opportunities to enhance the library to better support parallelization. As observed in the example of FEM matrix assembly, the achieved speedup is limited primarily not by the capabilities or efficiency of the `threadpool`, but by the algorithm itself, which is only partially parallelizable. A significant increase in speedup could be obtained if, instead of having each worker compute partial contributions (the element triples) and later sequentially assemble them, each worker were able to write directly into a dedicated sparse matrix structure designed for concurrent insertion. To make such a structure thread-safe without enforcing full serialization, the memory associated with each matrix row must be pre-allocated. Since the global matrix is sparse, the goal is not to reserve space for all possible entries but only for those that will actually be nonzero. This is feasible because, in finite element assembly, each row corresponds to a degree of freedom, and the number of nonzero entries in that row is determined solely by the mesh topology: specifically, by the number of basis functions whose support intersects that of the basis function associated with the row's degree of freedom. In structured meshes, this upper bound is easily known in advance, allowing the required memory for each matrix row to be pre-allocated. The resulting per-row capacity is an upper bound on the true number of nonzero elements, but it is typically tight enough to prevent significant over-allocation. For example, referring to the case illustrated in 3.2.4, for a structured triangular mesh with first-order polynomial nodal basis functions, the number of nonzero entries per row corresponds to the number of vertices adjacent to the vertex associated with the nodal function of that row. Therefore, the maximum number of nonzero entries per row is 7 (6 adjacent vertices plus the vertex itself), as illustrated in Figure 5.1:

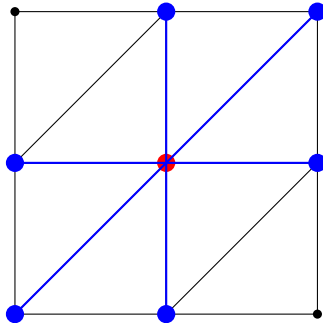


Figure 5.1: A simple structured triangular mesh.

Once concurrent writing to the sparse matrix is enabled, however, updates to the matrix must still be synchronized to avoid data races, since each worker processes mesh cells

independently and may attempt to update the same matrix entry as another worker. Several strategies can be adopted: a simple approach, proposed in [2], is to use mutexes to serialize updates, either a single global mutex protecting the entire matrix or a set of mutexes assigned to blocks of rows. A more sophisticated alternative, suggested in [2, 3], is to use coloring algorithms to partition mesh cells into groups that do not share degrees of freedom, ensuring that workers can safely assemble contributions without locking. This allows each group to be processed safely in parallel, as no two workers can modify the same matrix entry simultaneously.

The implementation of this strategy, in addition to integrating the coloring algorithm into the library for maximum efficiency, would also require a redesign of the mesh creation process. This would ensure that the partitioning of cells, and consequently the storage of their information in memory, is as cache-friendly as possible for parallel execution.

Bibliography

- [1] Eleonora Arnone, Aldo Clemente, Laura M. Sangalli, Eardi Lila, Jim Ramsay, and Luca Formaggia. fdapde: Physics informed spatial and functional data analysis, 2025. URL <https://cran.r-project.org/package=fdaPDE>. R package version available from CRAN.
- [2] Michal Boanský and Boek Patzák. Parallelization of assembly operation in finite element method. *Acta Polytechnica*, 2020.
- [3] Martin Kronbichler Bruno Turcksin and Wolfgang Bangerth. Workstream a design pattern for multicore-enabled finite element computations. *ACM Transactions on Mathematical Software*, 2016. doi: 10.1145/2851488.
- [4] P. G. Ciarlet. *The Finite Element Method for Elliptic Problems*. SIAM, Philadelphia, 2002.
- [5] cppreference.com. std::memory_order. https://en.cppreference.com/w/cpp/atomic/memory_order, . Accessed: 30-10-2025.
- [6] cppreference.com. Mutex functions: lock, unlock, try_lock, etc. <https://en.cppreference.com/w/cpp/thread/mutex/lock>, . Accessed: 30-10-2025.
- [7] Charles Frasch. Single producer single consumer lockfree waitfree fifo: From the ground up. Presentation at CppCon 2023, slides available at https://github.com/CppCon/CppCon2023/blob/main/Presentations/SPSC_Lock-free_Wait-Free_Fifo_from_the_Ground_Up_CppCon_2023.pdf, 2023.
- [8] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, an imprint of Elsevier, Waltham, MA, 5 edition, 2012. ISBN 9780123838728.
- [9] HPC Wiki. Scaling. <https://hpc-wiki.info/hpc/Scaling>. Accessed: 12-11-2025.
- [10] Alessandro Palummo, Eleonora Arnone, Luca Formaggia, and Laura M. Sangalli. Functional principal component analysis for incomplete space–time data. *Statistics and Computing*, 2024. Published online: 16 March 2024.

- [11] Alfio Quarteroni. *Numerical Models for Differential Problems*, volume 16. Springer, Cham, 2017.
- [12] J. O. Ramsay and B. W. Silverman. *Functional Data Analysis*. Springer, New York, 2 edition, 2005.
- [13] Laura M. Sangalli. Spatial regression with partial differential equation regularisation. *International Statistical Review*, 2021.
- [14] Anthony Williams. *C++ Concurrency in Action: Practical Multithreading*. Manning Publications Co., Shelter Island, NY, 1 edition, 2012. ISBN 9781933988771. Development editor: Cynthia Kane; Technical proofreader: Jonathan Wakely.